1.NUMPY PROGRAMS

1(i)Creating arrays:

Aim:

To create array in python using numpy header file.

Algorithm:

Step 1: Start the program.

Step 2: Import the NumPy library using import numpy as np.

Step 3: Create and display a 1D array using np.array([1, 2, 3, 4, 5]).

Step 4: Create and display a 2D array using np.array([[1, 2, 3], [4, 5, 6]]).

Step 5: Create and display a 3 × 3 matrix of zeros using np.zeros((3, 3)).

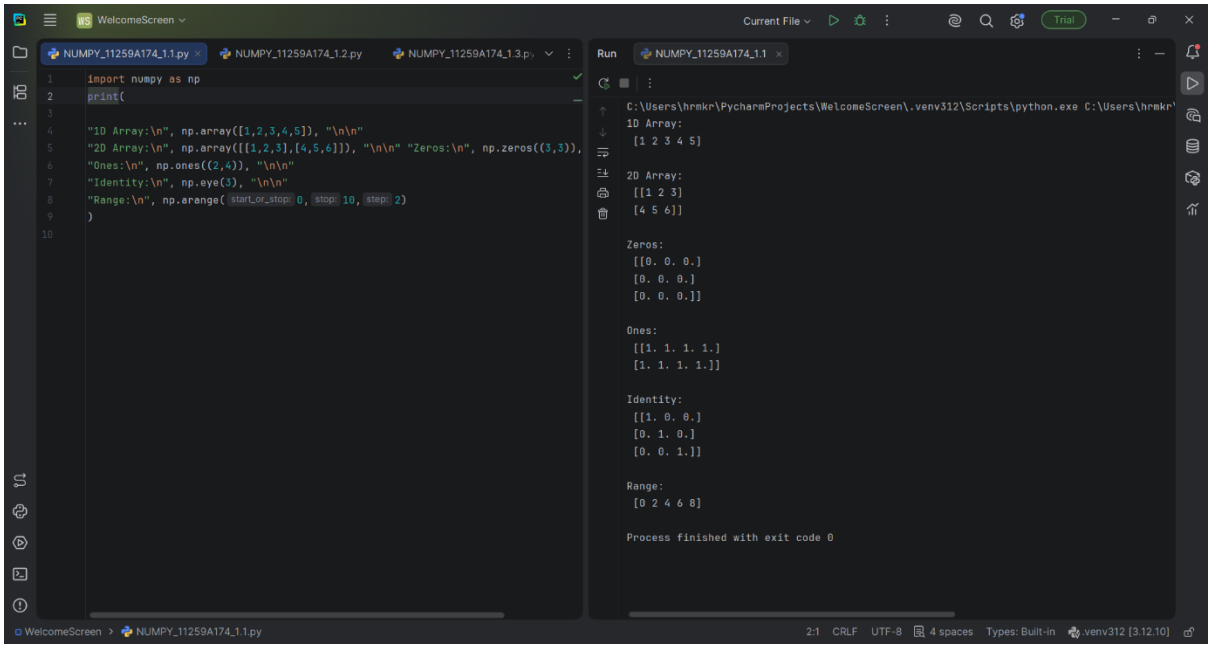
Step 6: Create and display a 2 × 4 matrix of ones using np.ones((2, 4)).

Step 7: Create and display a 3 × 3 identity matrix using np.eye(3).

Step 8: Generate and display numbers from 0 to 8 with a step size of 2 using np.arange(0, 10, 2).

Step 9: Stop the program.

Coding and Output:



Result:

The given program has been executed and output has been found out.

1(ii)Array operation:

Aim:

To perform arithmetic and mathematical operations on NumPy arrays, including addition, element-wise multiplication, scalar multiplication, square root, mean, and dot product.

Algorithm:

Step 1: Start the program.

Step 2: Import the NumPy library using import numpy as np.

Step 3: Create two NumPy arrays, a = [1, 2, 3] and b = [4, 5, 6].

Step 4: Display both arrays.

Step 5: Perform element-wise addition using a + b and display the result.

Step 6: Perform element-wise multiplication using a * b and display the result.

Step 7: Multiply each element of array a by 10 using a * 10 and display the result.

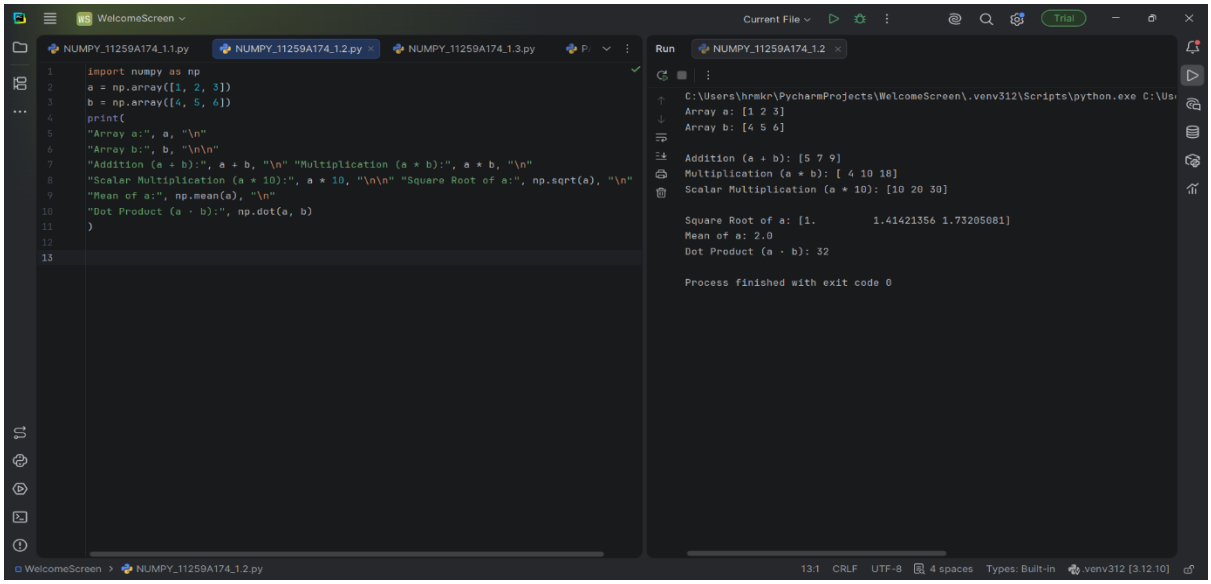
Step 8: Calculate the square root of each element in array a using np.sqrt(a) and display the result.

Step 9: Calculate the mean of array a using np.mean(a) and display the result.

Step 10: Calculate the dot product of arrays a and b using np.dot(a, b) and display the result.

Step 11: Stop the program.

Coding and output:



Result: The given program has been executed and output has been found out.

1(iii)Indexing and Slicing:

Aim:

To create a two-dimensional NumPy matrix and perform indexing and slicing operations to access a specific element, a row, a column, and a sub-matrix.

Algorithm:

Step 1: Start the program.

Step 2: Import the NumPy library using import numpy as np.

Step 3: Create a 3 × 3 matrix using np.array() with the elements 10, 20, 30, 40, 50, 60, 70, 80, and 90.

Step 4: Display the entire matrix.

Step 5: Access and display the element at index [1, 2] using matrix[1, 2].

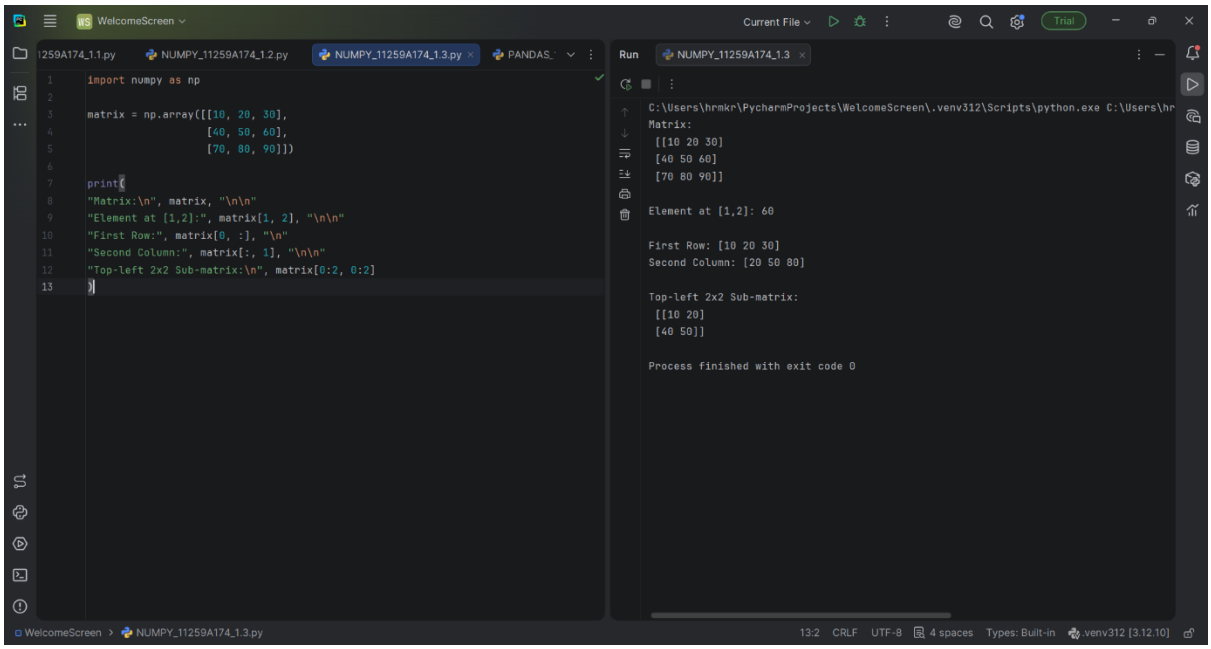
Step 6: Access and display the first row using matrix[0, :].

Step 7: Access and display the second column using matrix[:, 1].

Step 8: Extract and display the top-left 2 × 2 sub-matrix using matrix[0:2, 0:2].

Step 9: Stop the program.

Coding and Output:



Result:

The given program has been executed and output has been found out

2.PANDAS PROGRAMS

2(i)Creating Dataframes:

Aim:

To create a Pandas DataFrame from a dictionary and display the first few rows, information about data types and non-null values, and the statistical summary of numerical data.

Algorithm:

Step 1: Start the program.

Step 2: Import the Pandas library using import pandas as pd.

Step 3: Create a dictionary containing the columns Name, Age, and City with their respective values.

Step 4: Convert the dictionary into a Pandas DataFrame using pd.DataFrame(data).

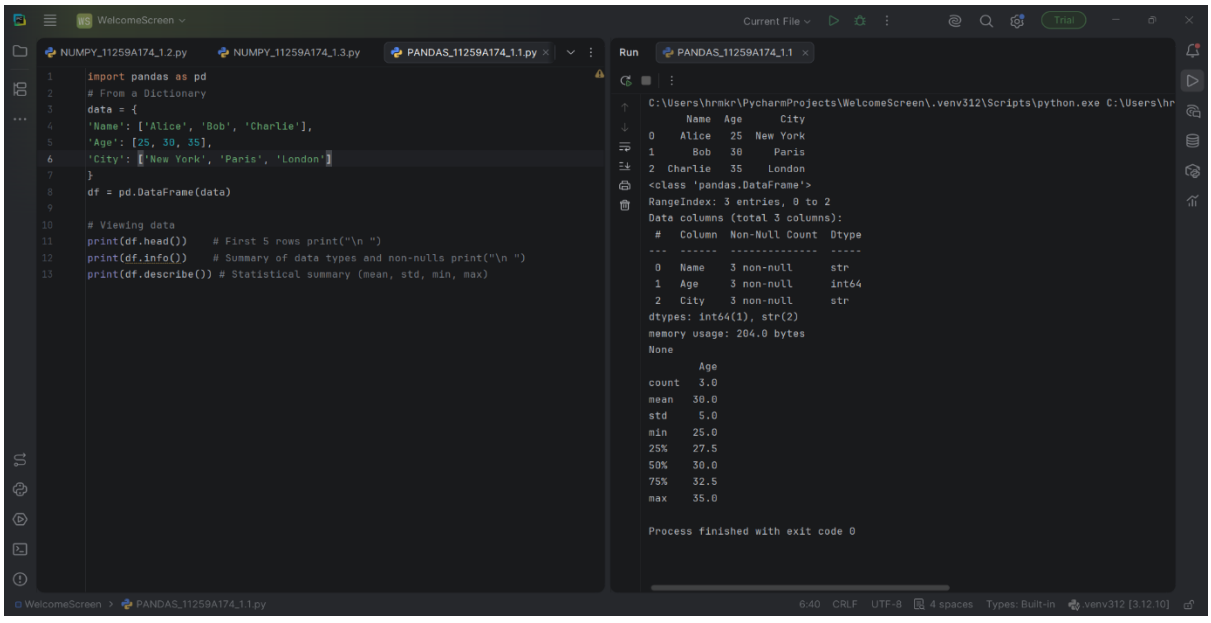
Step 5: Display the first five rows of the DataFrame using df.head().

Step 6: Display the DataFrame information, including column names, data types, and non-null counts, using df.info().

Step 7: Display the statistical summary of numerical columns using df.describe().

Step 8: Stop the program.

Coding and output:



Result:

The given program has been executed and output has been found out.

2(ii)Selecting and Filtering:

Aim:

To perform data selection and filtering operations on a Pandas DataFrame using column selection, conditional filtering, iloc (integer-based indexing), and loc (label-based indexing).

Algorithm:

Step 1: Start the program.

Step 2: Select the Age column from the DataFrame using df['Age'] and display it.

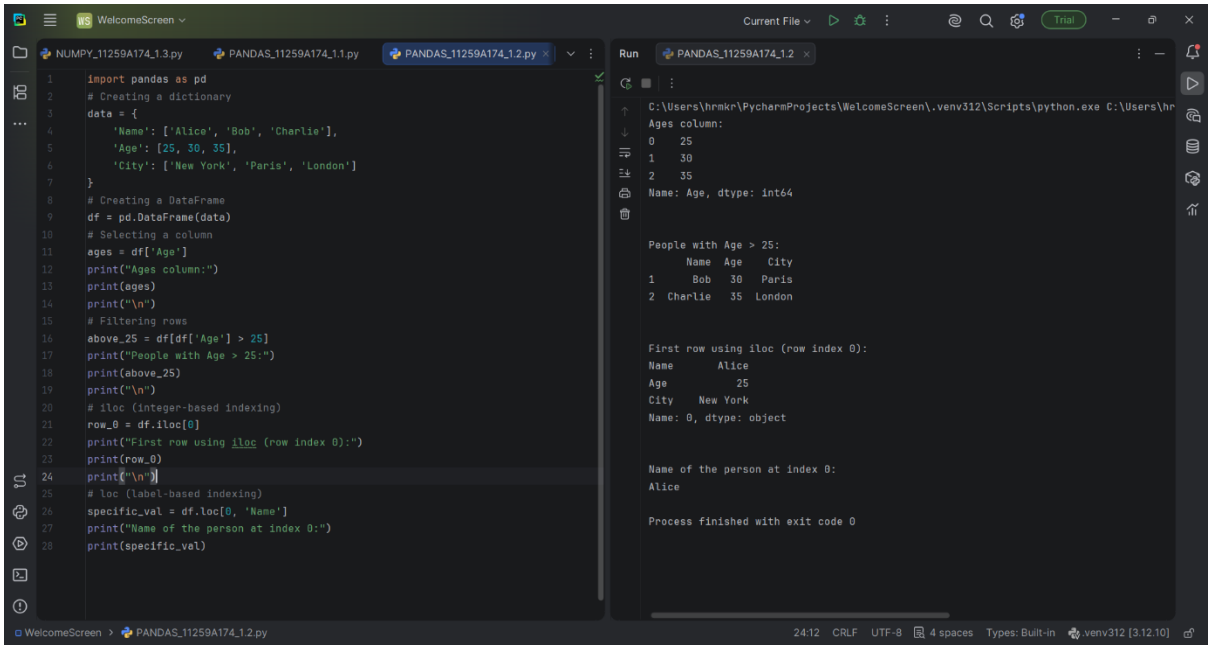
Step 3: Filter the rows where the age is greater than 25 using df[df['Age'] > 25] and display the results.

Step 4: Access the first row of the DataFrame using integer-based indexing with df.iloc[0] and display it.

Step 5: Access the Name value at index 0 using label-based indexing with df.loc[0, 'Name'] and display the result.

Step 6: Stop the program.

Coding and Output:



Result: The given program has been executed and output has been found out.

2(iii) Data Cleaning (Handling Missing Values):

Aim:

To identify and handle missing values in a Pandas DataFrame using isnull(), dropna(), and fillna(), including replacing missing age values with the mean of the Age column.

Algorithm:

Step 1: Start the program.

Step 2: Import the Pandas library as pd and the NumPy library as np.

Step 3: Create a sample dataset containing the columns Name, Age, and City, with some missing values represented by np.nan.

Step 4: Convert the dataset into a Pandas DataFrame using pd.DataFrame(data).

Step 5: Display the original DataFrame.

Step 6: Identify and count the missing values in each column using df.isnull().sum().

Step 7: Remove all rows containing missing values using df.dropna() and display the cleaned DataFrame.

Step 8: Replace all missing values with 0 using df.fillna(0) and display the updated DataFrame.

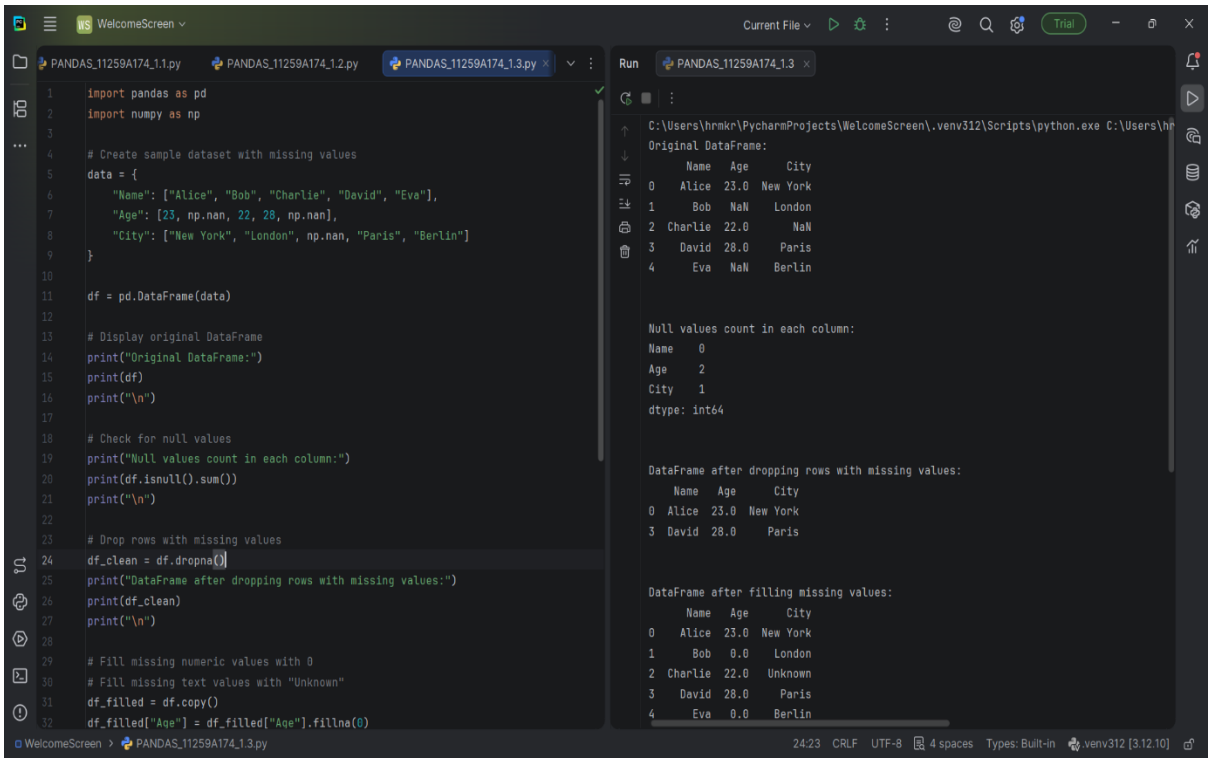
Step 9: Create a copy of the original DataFrame using df.copy().

Step 10: Calculate the mean of the Age column and replace its missing values with the calculated mean using fillna().

Step 11: Display the DataFrame after filling the missing age values with the mean.

Step 12: Stop the program.

Coding and Output:



Result:

The given program has been executed and output has been found out.

ExNO:3 Data Preprocessing

Aim

To load a dataset from a CSV file, check and handle missing values, and normalize numerical features using the StandardScaler class from the Scikit-learn library.

Algorithm:

Step 1: Start the program.

Step 2: Import the Pandas library and StandardScaler from sklearn.preprocessing.

Step 3: Load the dataset from data.csv using pd.read_csv().

Step 4: Display the first five rows of the original dataset using data.head().

Step 5: Check and display the number of missing values in each column using data.isnull().sum().

Step 6: Fill missing values in numerical columns with their respective column means using data.fillna(data.mean(numeric_only=True), inplace=True).

Step 7: Select the numerical columns Hours_Studied, Attendance, Assignments, and Marks as input features.

Step 8: Create a StandardScaler object to normalize the selected numerical data.

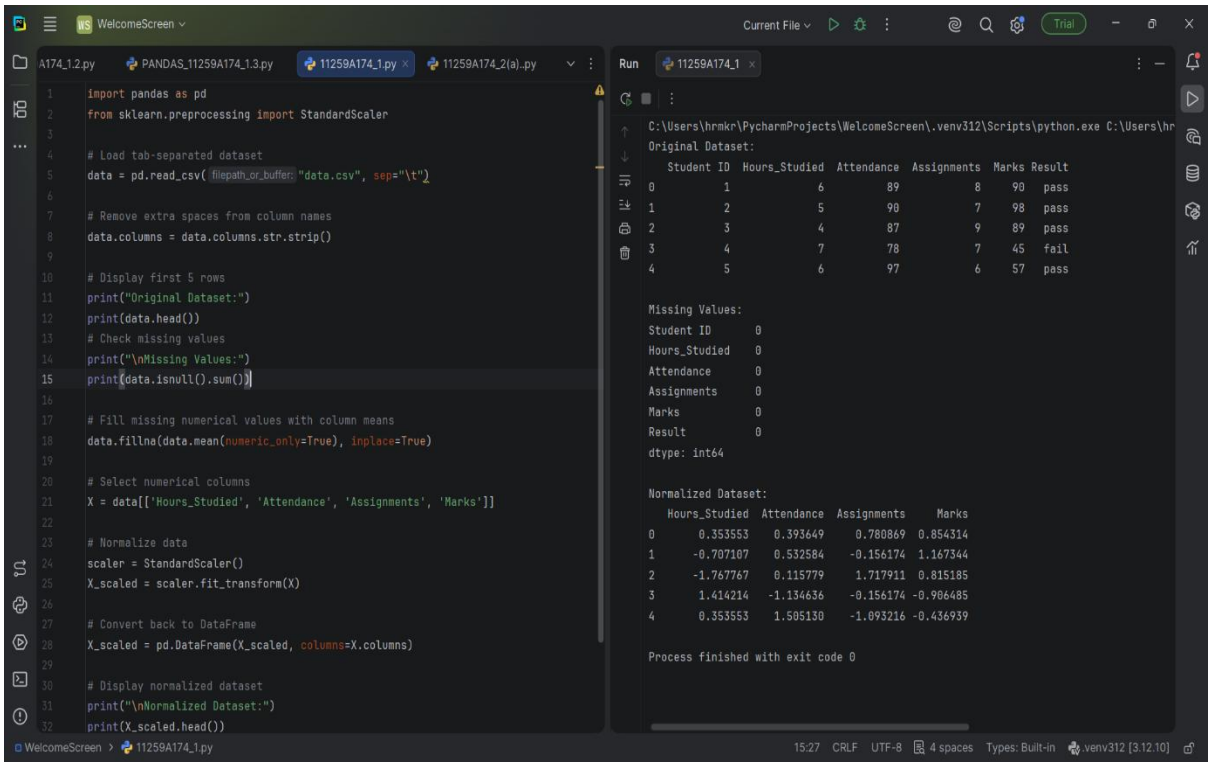
Step 9: Apply scaler.fit_transform(X) to standardize the features so that each column has approximately zero mean and unit variance.

Step 10: Convert the normalized array back into a Pandas DataFrame using pd.DataFrame() and retain the original column names.

Step 11: Display the first five rows of the normalized dataset.

Step 12: Stop the program.

Coding and output:



```
import pandas as pd
from sklearn.preprocessing import StandardScaler

# Load tab-separated dataset
data = pd.read_csv( filepath_or_buffer: "data.csv", sep="\t")

# Remove extra spaces from column names
data.columns = data.columns.str.strip()

# Display first 5 rows
print("Original Dataset:")
print(data.head())
# Check missing values
print("\nMissing Values:")
print(data.isnull().sum())

# Fill missing numerical values with column means
data.fillna(data.mean(numeric_only=True), inplace=True)

# Select numerical columns
X = data[['Hours_Studied', 'Attendance', 'Assignments', 'Marks']]

# Normalize data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Convert back to DataFrame
X_scaled = pd.DataFrame(X_scaled, columns=X.columns)

# Display normalized dataset
print("\nNormalized Dataset:")
print(X_scaled.head())
```

```
C:\Users\hrmkr\PycharmProjects\WelcomeScreen\.venv312\Scripts\python.exe C:\Users\hr
Original Dataset:
   Student ID  Hours_Studied  Attendance  Assignments  Marks Result
0           1              6          89            8     90   pass
1           2              5          90            7     98   pass
2           3              4          87            9     89   pass
3           4              7          78            7     45   fail
4           5              6          97            6     57   pass

Missing Values:
Student ID       0
Hours_Studied    0
Attendance       0
Assignments      0
Marks            0
Result           0
dtype: int64

Normalized Dataset:
   Hours_Studied  Attendance  Assignments     Marks
0       0.353553    0.393649     0.780869  0.854314
1      -0.707107    0.532584    -0.156174  1.167344
2      -1.767767    0.115779     1.717911  0.815185
3       1.414214   -1.134636    -0.156174 -0.906485
4       0.353553    1.505130    -1.093216 -0.436939

Process finished with exit code 0
```

Result:

The given program has been executed and output has been found out.

Exno:4 Data Split up and Cross validation.

A.DATA SPLITUP:

Aim:

To split a dataset into training and testing sets using the train_test_split() function from Scikit-learn, with 80% of the data used for training and 20% used for testing.

Algorithm

Step 1: Start the program.

Step 2: Import the Pandas library and train_test_split from sklearn.model_selection.

Step 3: Load the dataset from the CSV file data.csv using pd.read_csv().

Step 4: Select the input features Hours_Studied, Attendance, and Assignments and store them in variable X.

Step 5: Select the target column Result and store it in variable y.

Step 6: Split the dataset into training and testing sets using train_test_split() with test_size=0.2 and random_state=42.

Step 7: Store the training features and target values in X_train and y_train, respectively.

Step 8: Store the testing features and target values in X_test and y_test, respectively.

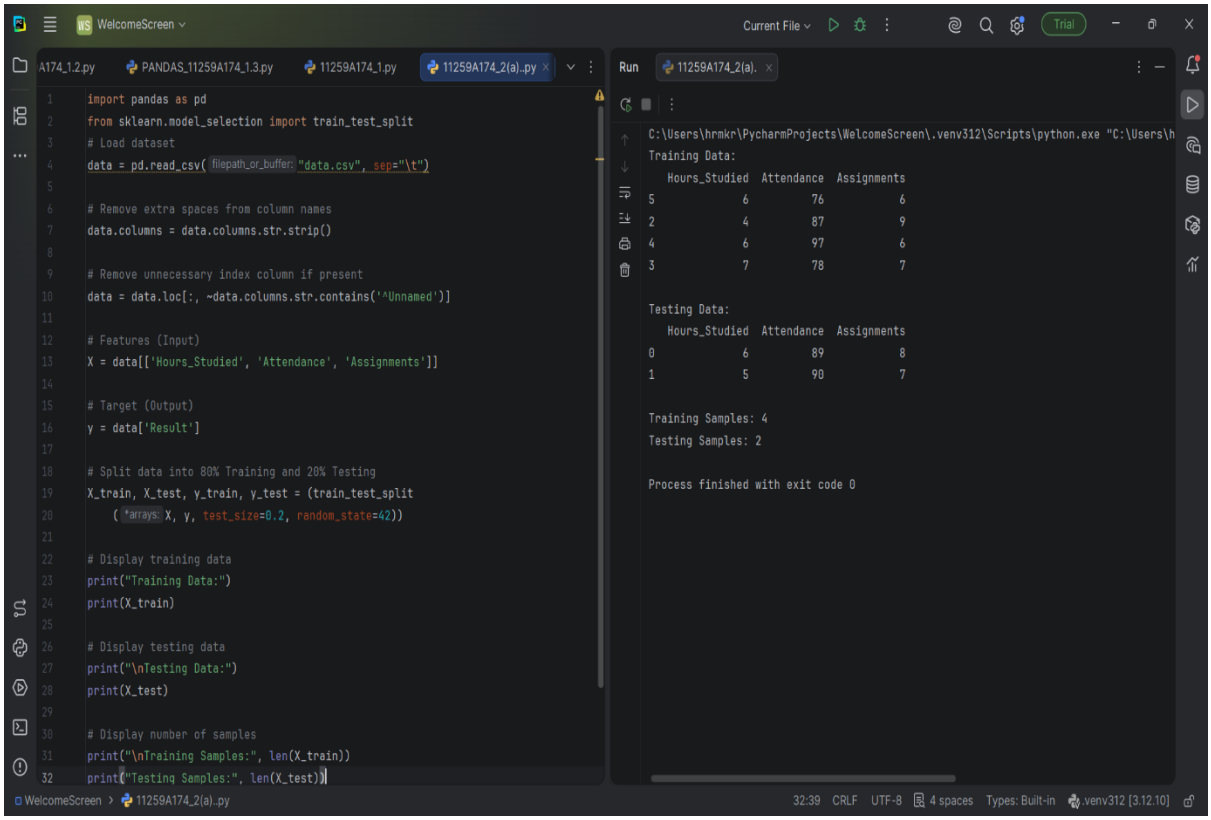
Step 9: Display the training data using X_train.

Step 10: Display the testing data using X_test.

Step 11: Display the number of training samples and testing samples using len(X_train) and len(X_test).

Step 12: Stop the program.

Coding and Output:



```
import pandas as pd
from sklearn.model_selection import train_test_split
# Load dataset
data = pd.read_csv( filepath_or_buffer: "data.csv", sep="\t")

# Remove extra spaces from column names
data.columns = data.columns.str.strip()

# Remove unnecessary index column if present
data = data.loc[:, ~data.columns.str.contains('^Unnamed')]

# Features (Input)
X = data[['Hours_Studied', 'Attendance', 'Assignments']]

# Target (Output)
y = data['Result']

# Split data into 80% Training and 20% Testing
X_train, X_test, y_train, y_test = (train_test_split
    ( *arrays: X, y, test_size=0.2, random_state=42))

# Display training data
print("Training Data:")
print(X_train)

# Display testing data
print("\nTesting Data:")
print(X_test)

# Display number of samples
print("\nTraining Samples:", len(X_train))
print("Testing Samples:", len(X_test))
```

```
C:\Users\hrmkr\PycharmProjects\WelcomeScreen\.venv312\Scripts\python.exe "C:\Users\h
Training Data:
   Hours_Studied  Attendance  Assignments
5              6          76            6
2              4          87            9
4              6          97            6
3              7          78            7

Testing Data:
   Hours_Studied  Attendance  Assignments
0              6          89            8
1              5          90            7

Training Samples: 4
Testing Samples: 2

Process finished with exit code 0
```

Result:

The given program has been executed and output has been found out.

B.DATA SPLITUP AND CROSS VALIDATION:

Aim

To implement the K-Nearest Neighbors (KNN) classification algorithm and evaluate its performance using 5-fold cross-validation to calculate individual accuracy scores and average accuracy.

Algorithm

Step 1: Start the program.

Step 2: Import the Pandas library and the required functions and classes: train_test_split, cross_val_score, and KNeighborsClassifier from Scikit-learn.

Step 3: Load the dataset from the CSV file data.csv using pd.read_csv().

Step 4: Select the input features Hours_Studied, Attendance, and Assignments and store them in X.

Step 5: Select the target column Result and store it in y.

Step 6: Split the dataset into training and testing sets using train_test_split() with 80% training data and 20% testing data.

Step 7: Display the number of training samples and testing samples.

Step 8: Create a KNN classification model using KNeighborsClassifier(n_neighbors=3), where \(K=3\).

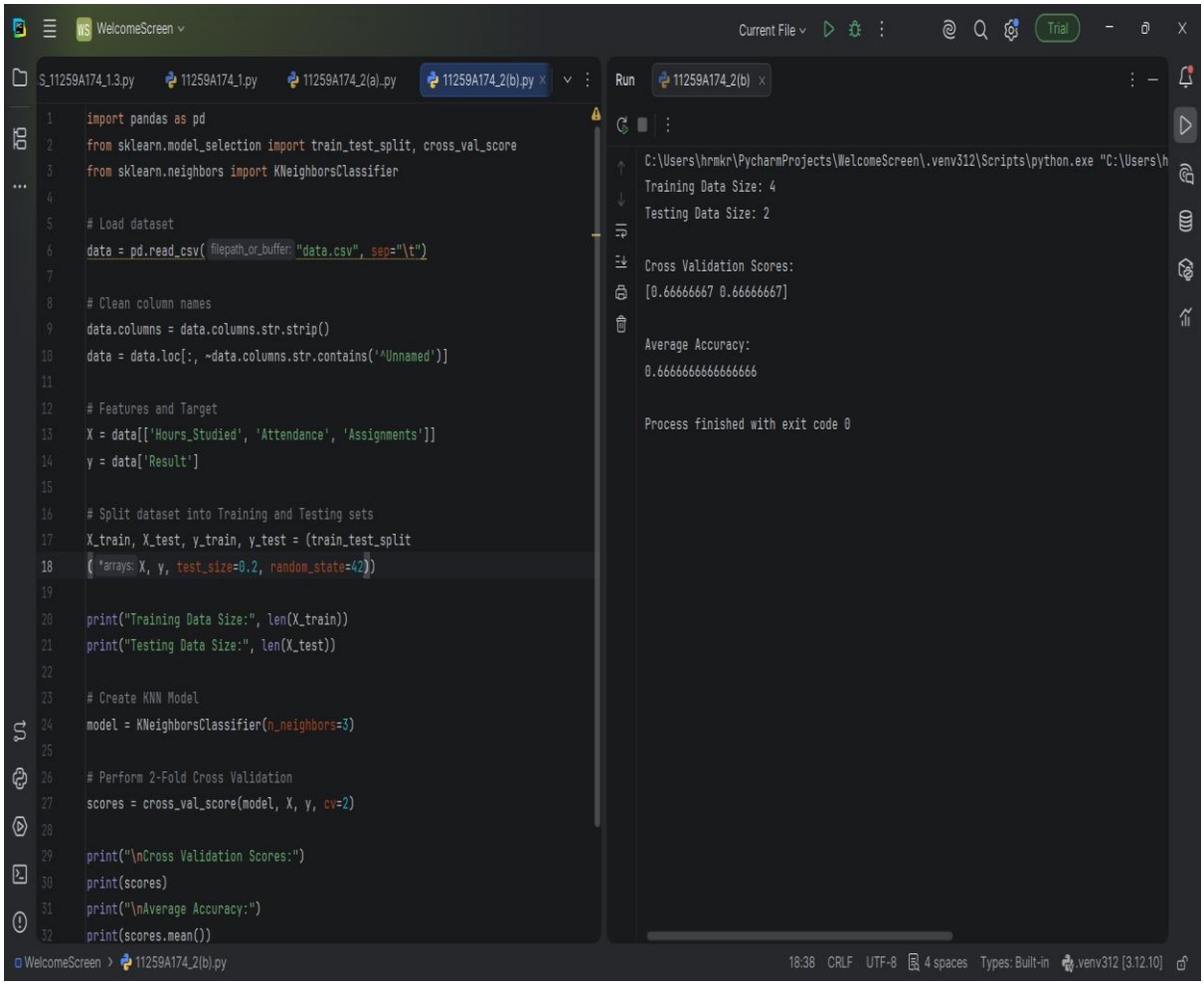
Step 9: Perform 5-fold cross-validation using cross_val_score(model, X, y, cv=5) to evaluate the model across five different data splits.

Step 10: Display the cross-validation accuracy scores.

Step 11: Calculate and display the average accuracy using scores.mean().

Step 12: Stop the program.

Coding and Output:



```
import pandas as pd
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.neighbors import KNeighborsClassifier

# Load dataset
data = pd.read_csv( filepath_or_buffer: "data.csv", sep="\t")

# Clean column names
data.columns = data.columns.str.strip()
data = data.loc[:, ~data.columns.str.contains('^Unnamed')]

# Features and Target
X = data[['Hours_Studied', 'Attendance', 'Assignments']]
y = data['Result']

# Split dataset into Training and Testing sets
X_train, X_test, y_train, y_test = (train_test_split
( *arrays: X, y, test_size=0.2, random_state=42))

print("Training Data Size:", len(X_train))
print("Testing Data Size:", len(X_test))

# Create KNN Model
model = KNeighborsClassifier(n_neighbors=3)

# Perform 2-Fold Cross Validation
scores = cross_val_score(model, X, y, cv=2)

print("\nCross Validation Scores:")
print(scores)
print("\nAverage Accuracy:")
print(scores.mean())
```

```
C:\Users\hrmkr\PycharmProjects\WelcomeScreen\.venv312\Scripts\python.exe "C:\Users\h
Training Data Size: 4
Testing Data Size: 2

Cross Validation Scores:
[0.66666667 0.66666667]

Average Accuracy:
0.6666666666666666

Process finished with exit code 0
```

Result:

The given program has been executed and output has been found out.

Exno:5 Implement Linear Regression

Aim

To implement a Simple Linear Regression model using Scikit-learn to predict students' marks based on the number of hours studied, evaluate the model using Mean Squared Error (MSE) and R^2 Score, and visualize the actual values and regression line using Matplotlib.

Algorithm

Step 1: Start the program.

Step 2: Import the required libraries: Pandas, Matplotlib, train_test_split, LinearRegression, mean_squared_error, and r2_score.

Step 3: Load the dataset from the CSV file data.csv using pd.read_csv().

Step 4: Select the Hours_Studied column as the input feature \(X\) and the Marks column as the target variable \(y\).

Step 5: Split the dataset into training and testing sets using train_test_split(), with 80% training data and 20% testing data.

Step 6: Create a Linear Regression model using LinearRegression().

Step 7: Train the model using the training data with model.fit(X_train, y_train).

Step 8: Predict the marks for the testing data using model.predict(X_test) and store the predictions in y_pred.

Step 9: Evaluate the model by calculating and displaying the Mean Squared Error (MSE) and R^2 Score.

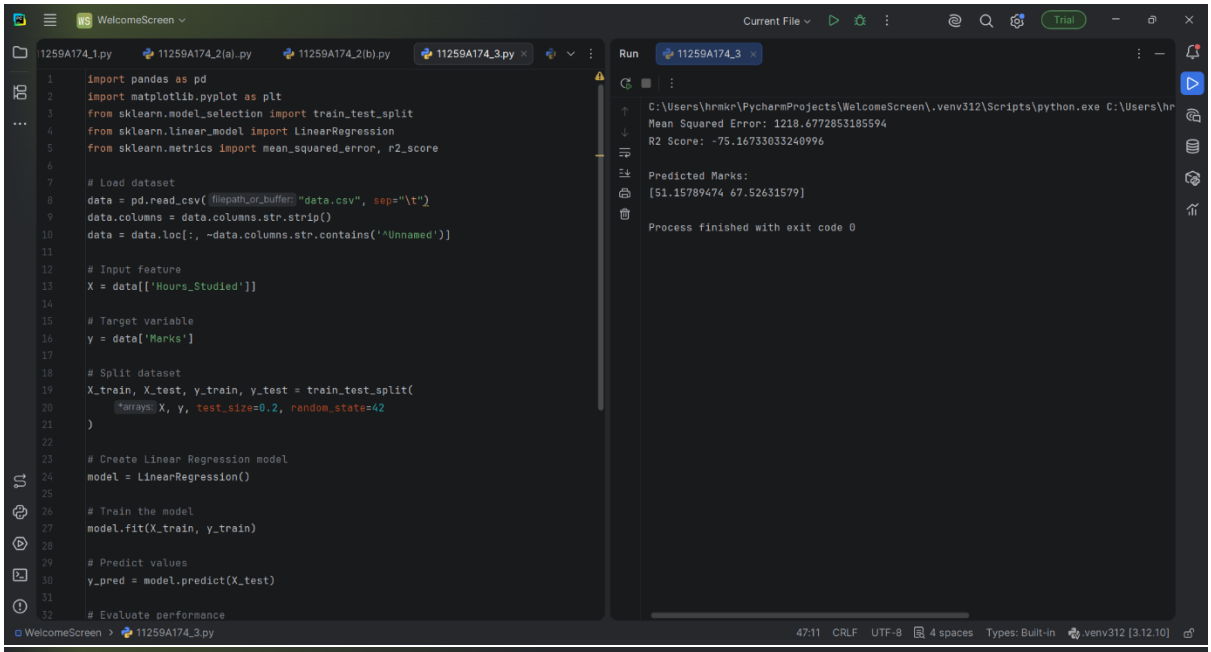
Step 10: Display the predicted marks.

Step 11: Plot the actual test data using a scatter plot and the predicted values using a regression line.

Step 12: Add labels to the X-axis and Y-axis, set the graph title, display the legend, and show the graph using plt.show().

Step 13: Stop the program.

Coding and Output:



```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Load dataset
data = pd.read_csv( filepath_or_buffer: "data.csv", sep="\t")
data.columns = data.columns.str.strip()
data = data.loc[:, ~data.columns.str.contains('^Unnamed')]

# Input feature
X = data[['Hours_Studied']]

# Target variable
y = data['Marks']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    *arrays: X, y, test_size=0.2, random_state=42
)

# Create Linear Regression model
model = LinearRegression()

# Train the model
model.fit(X_train, y_train)

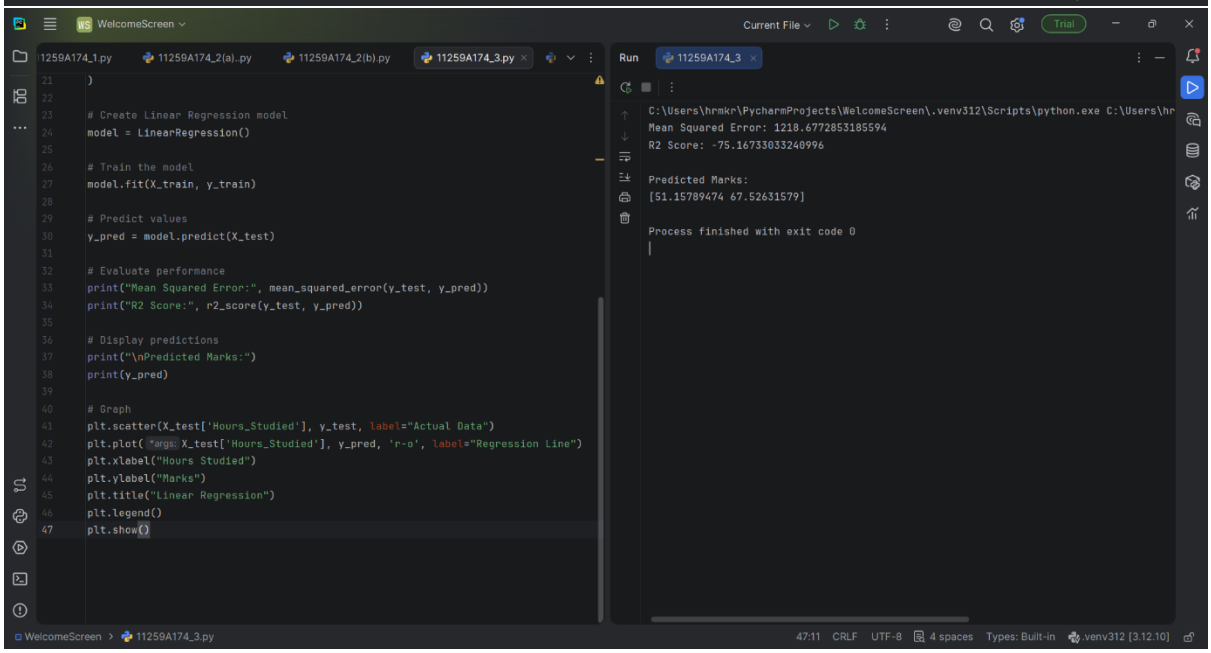
# Predict values
y_pred = model.predict(X_test)

# Evaluate performance
```

```
C:\Users\hrmkr\PycharmProjects\WelcomeScreen\.venv312\Scripts\python.exe C:\Users\hr
Mean Squared Error: 1218.6772853185594
R2 Score: -75.16733033240996

Predicted Marks:
[51.15789474 67.52631579]

Process finished with exit code 0
```



```
)

# Create Linear Regression model
model = LinearRegression()

# Train the model
model.fit(X_train, y_train)

# Predict values
y_pred = model.predict(X_test)

# Evaluate performance
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))

# Display predictions
print("\nPredicted Marks:")
print(y_pred)

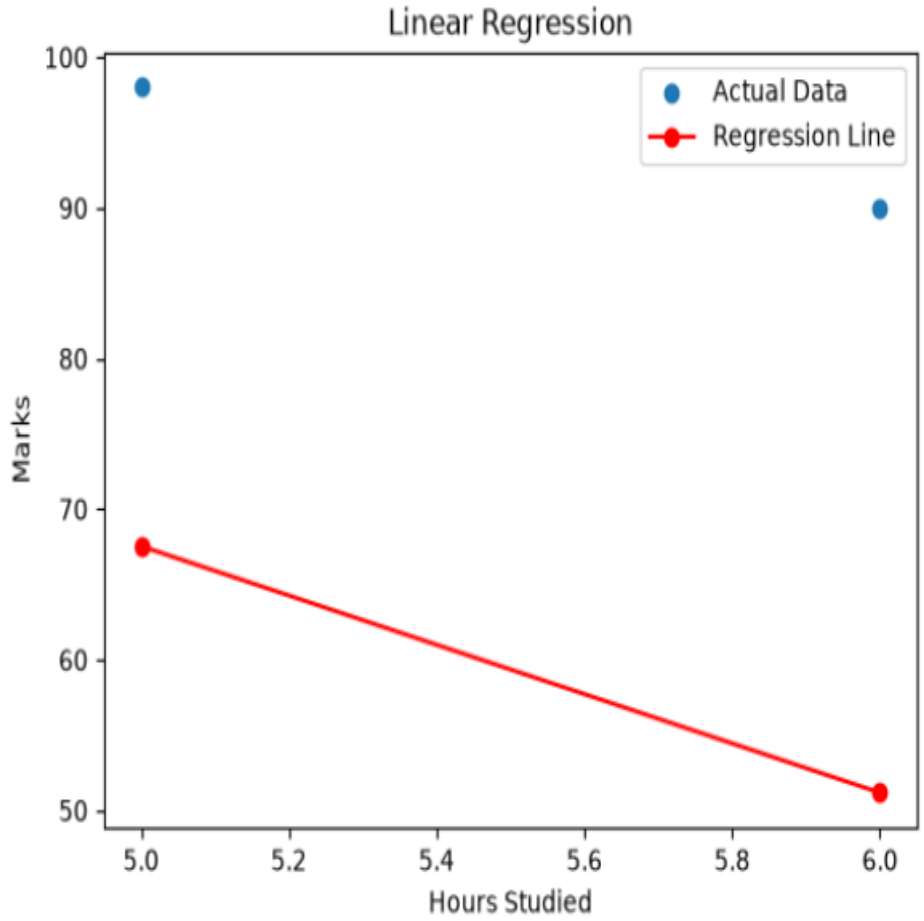
# Graph
plt.scatter(X_test['Hours_Studied'], y_test, label="Actual Data")
plt.plot( *args: X_test['Hours_Studied'], y_pred, 'r-o', label="Regression Line")
plt.xlabel("Hours Studied")
plt.ylabel("Marks")
plt.title("Linear Regression")
plt.legend()
plt.show()
```

```
C:\Users\hrmkr\PycharmProjects\WelcomeScreen\.venv312\Scripts\python.exe C:\Users\hr
Mean Squared Error: 1218.6772853185594
R2 Score: -75.16733033240996

Predicted Marks:
[51.15789474 67.52631579]

Process finished with exit code 0
```

Graph:



Result:

The given program has been executed and output has been found out.

Exno:6 Implement Classification Model Using KNN and Compute Evaluation Metrics

Aim

To implement the K-Nearest Neighbors (KNN) classification algorithm using Scikit-learn to predict students' results based on hours studied, attendance, and assignments, and evaluate the model using accuracy, confusion matrix, and classification report.

Algorithm

Step 1: Start the program.

Step 2: Import the required libraries: Pandas, train_test_split, KNeighborsClassifier, accuracy_score, confusion_matrix, and classification_report.

Step 3: Load the dataset from the CSV file data.csv using pd.read_csv().

Step 4: Select Hours_Studied, Attendance, and Assignments as input features and store them in X.

Step 5: Select the Result column as the target variable and store it in y.

Step 6: Split the dataset into training and testing sets using train_test_split(), with 80% training data and 20% testing data.

Step 7: Create a KNN classification model using KNeighborsClassifier(n_neighbors=3).

Step 8: Train the model using the training data with model.fit(X_train, y_train).

Step 9: Predict the results for the testing data using model.predict(X_test).

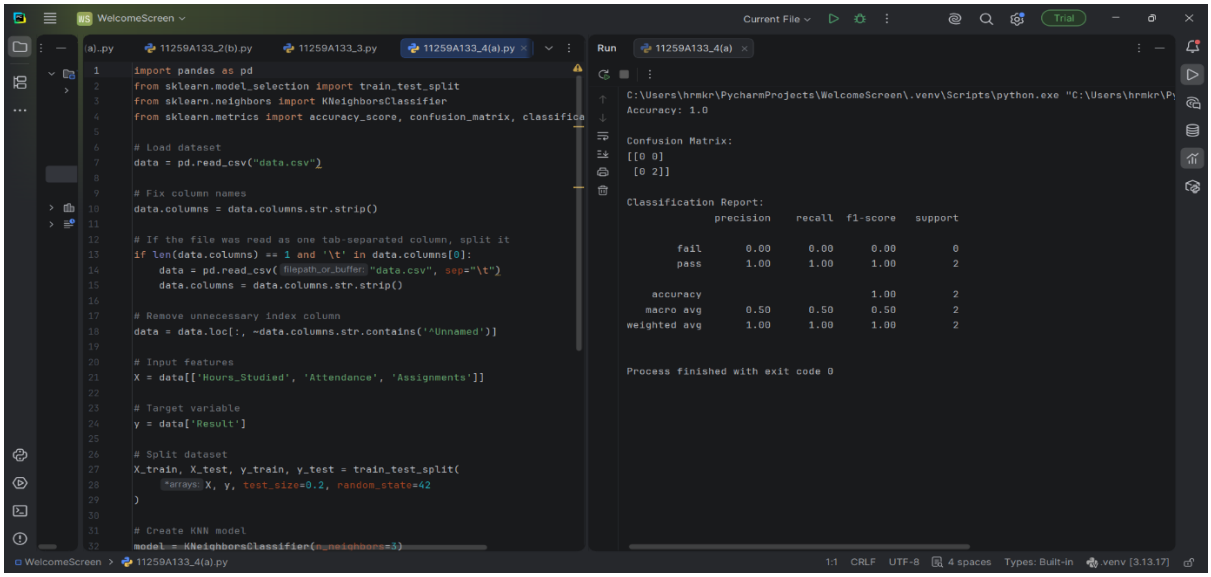
Step 10: Calculate and display the model's accuracy using accuracy_score(y_test, y_pred).

Step 11: Generate and display the confusion matrix using confusion_matrix(y_test, y_pred).

Step 12: Generate and display the classification report using classification_report(y_test, y_pred), which includes precision, recall, F1-score, and support.

Step 13: Stop the program.

Coding and Output:



```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classifica

# Load dataset
data = pd.read_csv("data.csv")

# Fix column names
data.columns = data.columns.str.strip()

# If the file was read as one tab-separated column, split it
if len(data.columns) == 1 and '\t' in data.columns[0]:
    data = pd.read_csv( filepath_or_buffer: "data.csv", sep="\t")
    data.columns = data.columns.str.strip()

# Remove unnecessary index column
data = data.loc[:, ~data.columns.str.contains('^Unnamed')]

# Input features
X = data[['Hours_Studied', 'Attendance', 'Assignments']]

# Target variable
y = data['Result']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    *arrays: X, y, test_size=0.2, random_state=42
)

# Create KNN model
model = KNeighborsClassifier(n_neighbors=3)
```

```
C:\Users\hrmkr\PycharmProjects\WelcomeScreen\.venv\Scripts\python.exe "C:\Users\hrmkr\P
Accuracy: 1.0

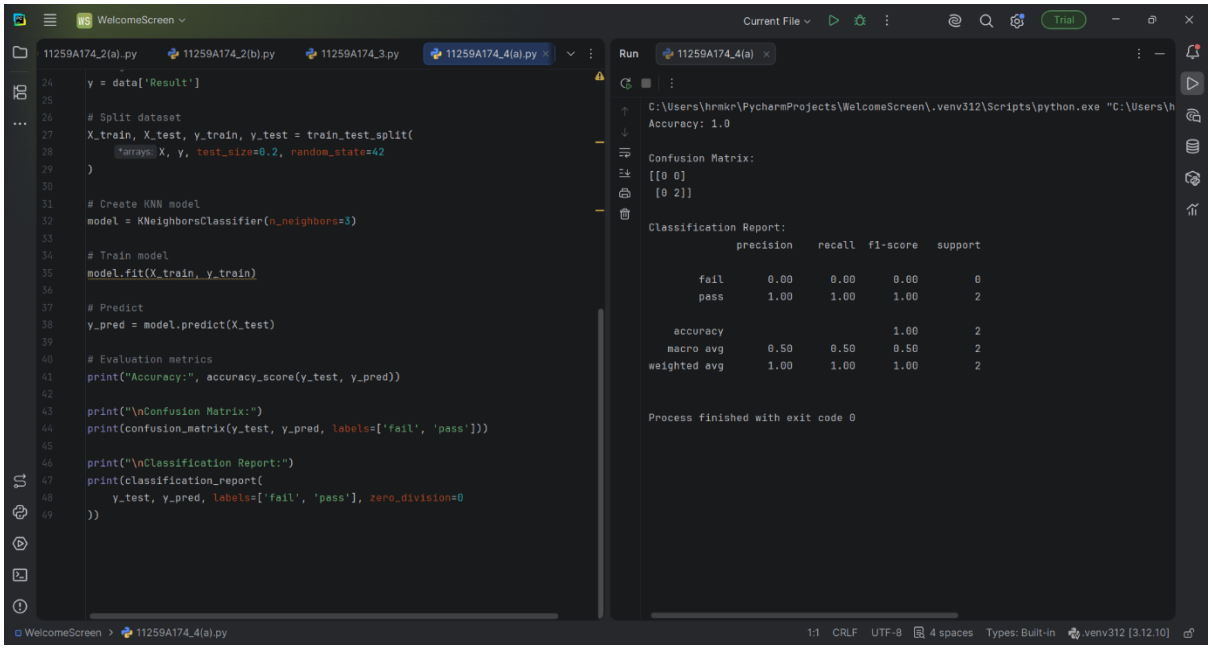
Confusion Matrix:
[[0 0]
 [0 2]]

Classification Report:
              precision    recall  f1-score   support

        fail       0.00      0.00      0.00         0
        pass       1.00      1.00      1.00         2

    accuracy                           1.00         2
   macro avg       0.50      0.50      0.50         2
weighted avg       1.00      1.00      1.00         2


Process finished with exit code 0
```



```
y = data['Result']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    *arrays: X, y, test_size=0.2, random_state=42
)

# Create KNN model
model = KNeighborsClassifier(n_neighbors=3)

# Train model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Evaluation metrics
print("Accuracy:", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred, labels=['fail', 'pass']))

print("\nClassification Report:")
print(classification_report(
    y_test, y_pred, labels=['fail', 'pass'], zero_division=0
))
```

```
C:\Users\hrmkr\PycharmProjects\WelcomeScreen\.venv312\Scripts\python.exe "C:\Users\h
Accuracy: 1.0

Confusion Matrix:
[[0 0]
 [0 2]]

Classification Report:
              precision    recall  f1-score   support

        fail       0.00      0.00      0.00         0
        pass       1.00      1.00      1.00         2

    accuracy                           1.00         2
   macro avg       0.50      0.50      0.50         2
weighted avg       1.00      1.00      1.00         2


Process finished with exit code 0
```

Result:

The given program has been executed and output has been found out.

6B Implement Classification Model Using SVM and Compute Evaluation Metrics

Aim

To implement a Support Vector Machine (SVM) classification algorithm using Scikit-learn to predict students' results based on hours studied, attendance, and assignments, and evaluate the model using accuracy, confusion matrix, and classification report.

Algorithm

Step 1: Start the program.

Step 2: Import the required libraries: Pandas, train_test_split, SVC, accuracy_score, confusion_matrix, and classification_report.

Step 3: Load the dataset from the CSV file data.csv using pd.read_csv().

Step 4: Select Hours_Studied, Attendance, and Assignments as input features and store them in X.

Step 5: Select the Result column as the target variable and store it in y.

Step 6: Split the dataset into training and testing sets using train_test_split(), with 80% training data and 20% testing data, and set random_state=42.

Step 7: Create an SVM classification model using SVC(kernel='linear') with a linear kernel.

Step 8: Train the SVM model using the training data with model.fit(X_train, y_train).

Step 9: Predict the results for the testing data using model.predict(X_test).

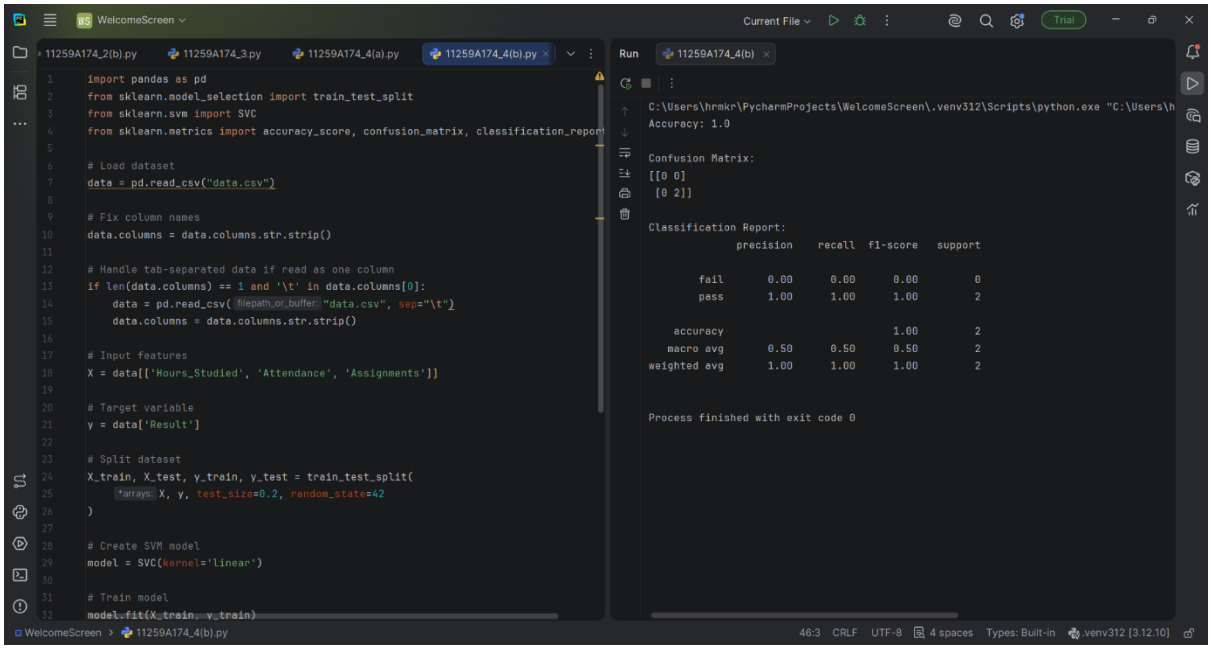
Step 10: Calculate and display the model's accuracy using accuracy_score(y_test, y_pred).

Step 11: Generate and display the confusion matrix using confusion_matrix(y_test, y_pred).

Step 12: Generate and display the classification report using classification_report(y_test, y_pred), including precision, recall, F1-score, and support.

Step 13: Stop the program.

Coding and algorithm:



```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load dataset
data = pd.read_csv("data.csv")

# Fix column names
data.columns = data.columns.str.strip()

# Handle tab-separated data if read as one column
if len(data.columns) == 1 and '\t' in data.columns[0]:
    data = pd.read_csv( filepath_or_buffer: "data.csv", sep="\t")
    data.columns = data.columns.str.strip()

# Input features
X = data[['Hours_Studied', 'Attendance', 'Assignments']]

# Target variable
y = data['Result']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    *arrays: X, y, test_size=0.2, random_state=42
)

# Create SVM model
model = SVC(kernel='linear')

# Train model
model.fit(X_train, y_train)
```

```
C:\Users\hrmkr\PycharmProjects\WelcomeScreen\.venv312\Scripts\python.exe "C:\Users\h
Accuracy: 1.0

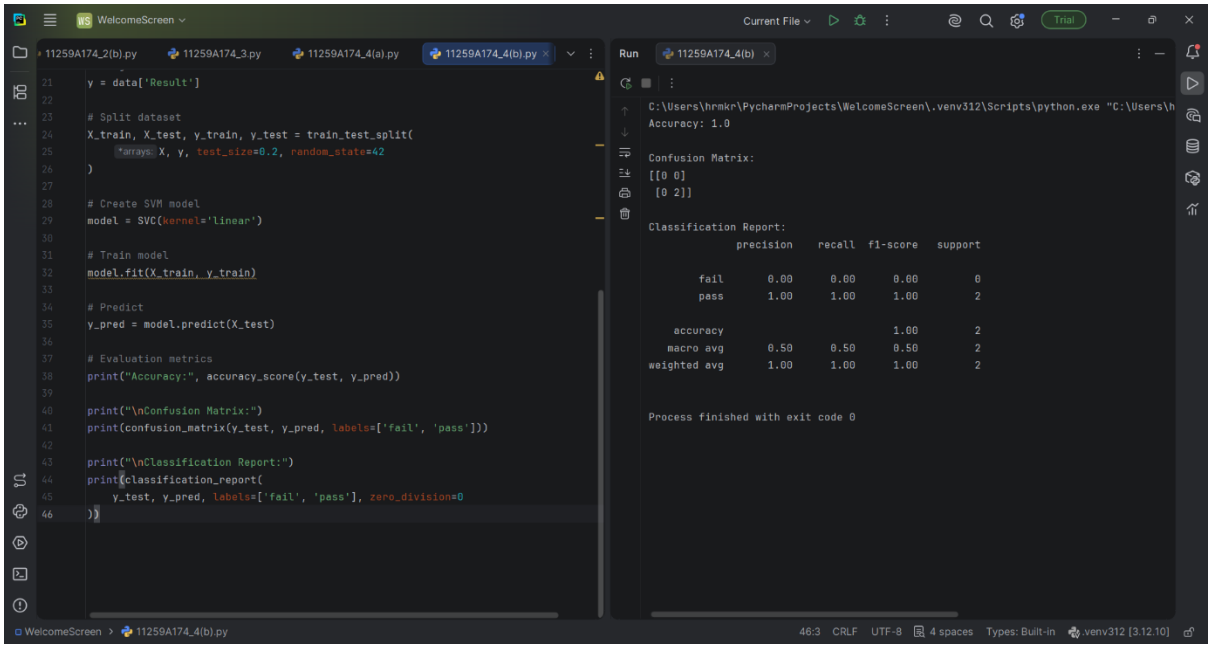
Confusion Matrix:
[[0 0]
 [0 2]]

Classification Report:
              precision    recall  f1-score   support

        fail       0.00      0.00      0.00         0
        pass       1.00      1.00      1.00         2

    accuracy                           1.00         2
   macro avg       0.50      0.50      0.50         2
weighted avg       1.00      1.00      1.00         2


Process finished with exit code 0
```



```
y = data['Result']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    *arrays: X, y, test_size=0.2, random_state=42
)

# Create SVM model
model = SVC(kernel='linear')

# Train model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Evaluation metrics
print("Accuracy:", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred, labels=['fail', 'pass']))

print("\nClassification Report:")
print(classification_report(
    y_test, y_pred, labels=['fail', 'pass'], zero_division=0
))
```

```
C:\Users\hrmkr\PycharmProjects\WelcomeScreen\.venv312\Scripts\python.exe "C:\Users\h
Accuracy: 1.0

Confusion Matrix:
[[0 0]
 [0 2]]

Classification Report:
              precision    recall  f1-score   support

        fail       0.00      0.00      0.00         0
        pass       1.00      1.00      1.00         2

    accuracy                           1.00         2
   macro avg       0.50      0.50      0.50         2
weighted avg       1.00      1.00      1.00         2


Process finished with exit code 0
```

Result:

The given program has been executed and output has been found out.

Exno7: Implement K-Means Clustering

Aim

To implement the K-Means Clustering algorithm using Scikit-learn to group students based on their hours studied and marks into three clusters, display the clustered data, and visualize the clusters and their centroids using Matplotlib.

Algorithm

Step 1: Start the program.

Step 2: Import the required libraries: Pandas, Matplotlib, and KMeans from sklearn.cluster.

Step 3: Load the dataset from the CSV file data.csv using pd.read_csv().

Step 4: Select the Hours_Studied and Marks columns as input features and store them in X.

Step 5: Create a K-Means clustering model with three clusters using KMeans(n_clusters=3, random_state=42).

Step 6: Train the model on the selected features using kmeans.fit(X).

Step 7: Obtain the cluster labels for each data point using kmeans.labels_.

Step 8: Add the cluster labels to the original dataset in a new column named Cluster.

Step 9: Display the dataset containing the assigned cluster labels.

Step 10: Plot the data points using a scatter plot, with hours studied on the X-axis, marks on the Y-axis, and different colors representing the clusters.

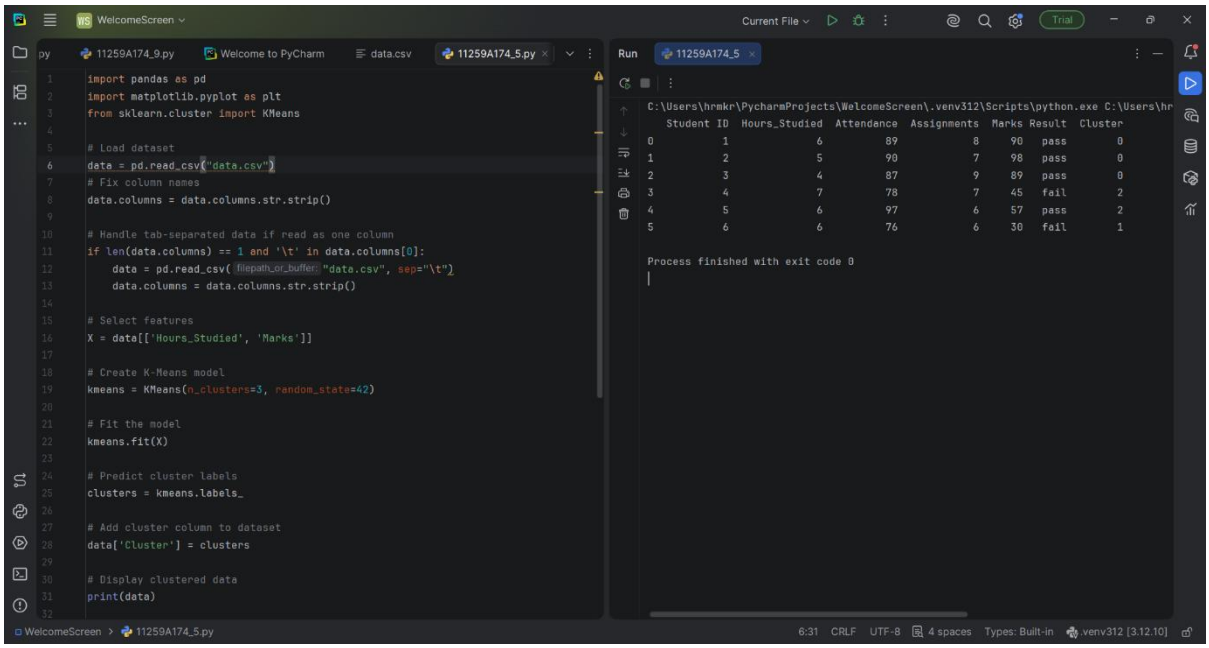
Step 11: Plot the cluster centroids using kmeans.cluster_centers_, with an X marker and a size of 200.

Step 12: Add the X-axis label, Y-axis label, graph title, and legend.

Step 13: Display the graph using plt.show().

Step 14: Stop the program.

Coding and Output:



```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Load dataset
data = pd.read_csv("data.csv")
# Fix column names
data.columns = data.columns.str.strip()

# Handle tab-separated data if read as one column
if len(data.columns) == 1 and '\t' in data.columns[0]:
    data = pd.read_csv( filepath_or_buffer: "data.csv", sep="\t")
    data.columns = data.columns.str.strip()

# Select features
X = data[['Hours_Studied', 'Marks']]

# Create K-Means model
kmeans = KMeans(n_clusters=3, random_state=42)

# Fit the model
kmeans.fit(X)

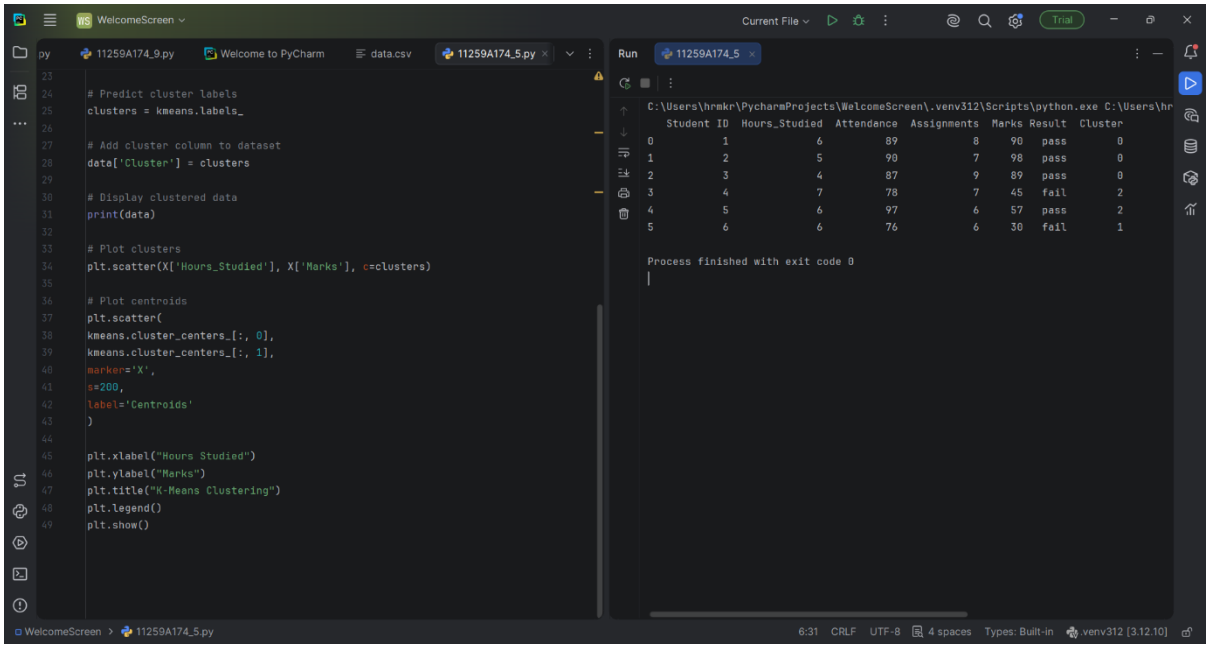
# Predict cluster labels
clusters = kmeans.labels_

# Add cluster column to dataset
data['Cluster'] = clusters

# Display clustered data
print(data)
```

```
C:\Users\hrmkr\PycharmProjects\WelcomeScreen\.venv312\Scripts\python.exe C:\Users\hr
   Student ID  Hours_Studied  Attendance  Assignments  Marks Result  Cluster
0           1              6          89            8     90   pass        0
1           2              5          90            7     98   pass        0
2           3              4          87            9     89   pass        0
3           4              7          78            7     45   fail        2
4           5              6          97            6     57   pass        2
5           6              6          76            6     30   fail        1

Process finished with exit code 0
```



```
# Predict cluster labels
clusters = kmeans.labels_

# Add cluster column to dataset
data['Cluster'] = clusters

# Display clustered data
print(data)

# Plot clusters
plt.scatter(X['Hours_Studied'], X['Marks'], c=clusters)

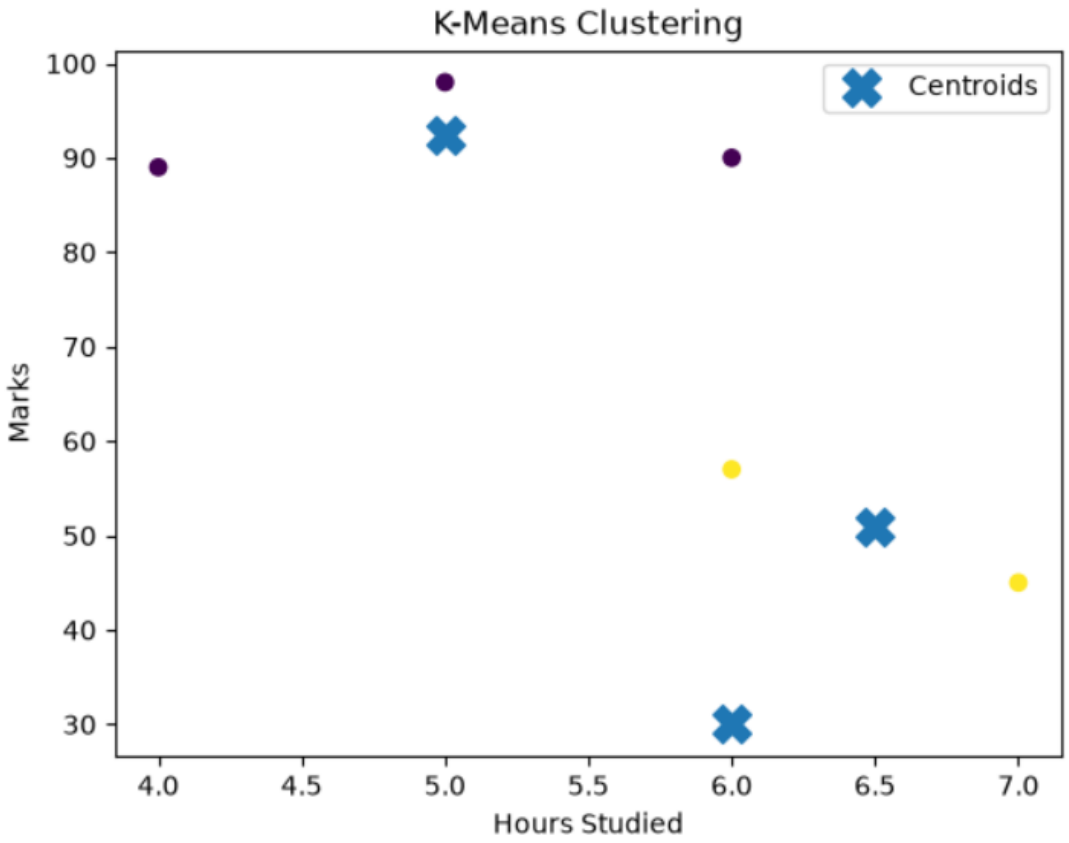
# Plot centroids
plt.scatter(
kmeans.cluster_centers_[:, 0],
kmeans.cluster_centers_[:, 1],
marker='X',
s=200,
label='Centroids'
)

plt.xlabel("Hours Studied")
plt.ylabel("Marks")
plt.title("K-Means Clustering")
plt.legend()
plt.show()
```

```
C:\Users\hrmkr\PycharmProjects\WelcomeScreen\.venv312\Scripts\python.exe C:\Users\hr
   Student ID  Hours_Studied  Attendance  Assignments  Marks Result  Cluster
0           1              6          89            8     90   pass        0
1           2              5          90            7     98   pass        0
2           3              4          87            9     89   pass        0
3           4              7          78            7     45   fail        2
4           5              6          97            6     57   pass        2
5           6              6          76            6     30   fail        1

Process finished with exit code 0
```

Graph:



Result:

The given program has been executed and output has been found out.

Exno:8 Implement Hierarchical Clustering

Aim

To implement Hierarchical Clustering using SciPy and Scikit-learn to group students based on their hours studied and marks into three clusters, visualize the hierarchical relationships using a dendrogram, and display the resulting clusters using a scatter plot.

Algorithm

Step 1: Start the program.

Step 2: Import the required libraries: Pandas, Matplotlib, dendrogram, linkage, and AgglomerativeClustering.

Step 3: Load the dataset from the CSV file data.csv using pd.read_csv().

Step 4: Select the Hours_Studied and Marks columns as input features and store them in X.

Step 5: Create a linkage matrix using linkage(X, method='ward') to calculate the hierarchical relationships between data points.

Step 6: Create a figure of size 8 × 5 and plot the dendrogram using dendrogram(linked).

Step 7: Add the title, X-axis label, and Y-axis label to the dendrogram, then display the graph using plt.show().

Step 8: Create an Agglomerative Clustering model with three clusters using AgglomerativeClustering(n_clusters=3).

Step 9: Apply the clustering algorithm to the selected features using hc.fit_predict(X) to obtain the cluster labels.

Step 10: Add the cluster labels to the original dataset in a new column named Cluster.

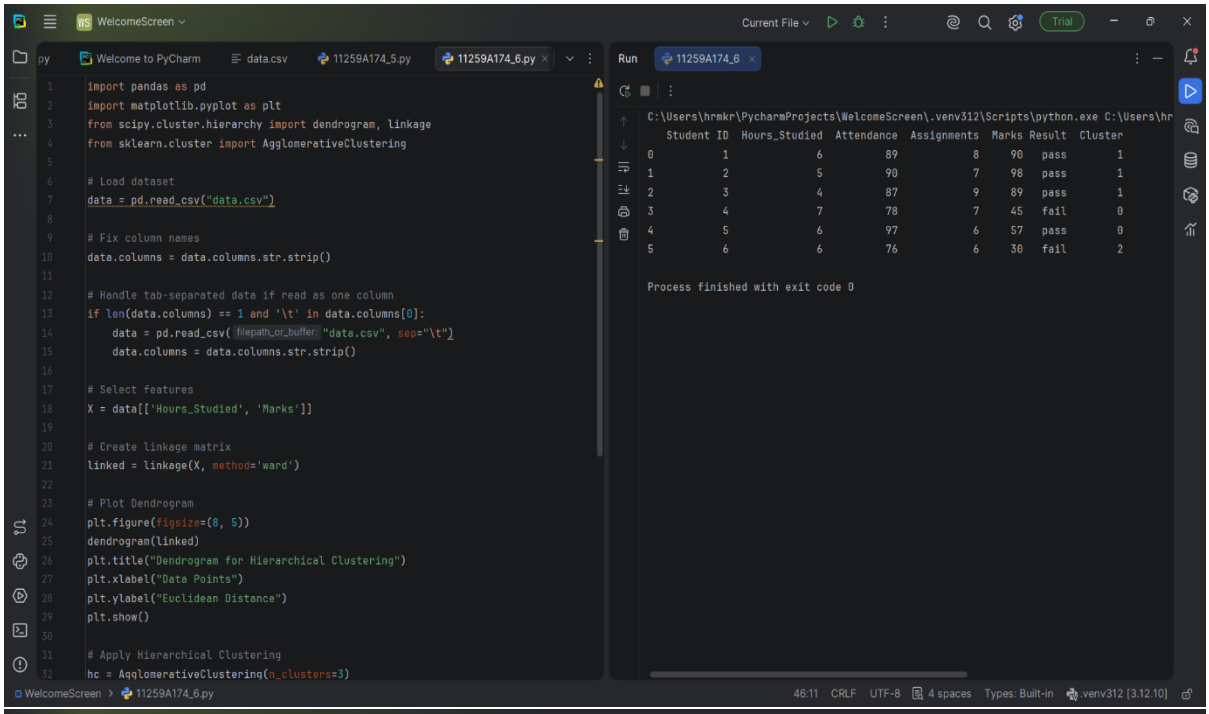
Step 11: Display the dataset with the assigned cluster labels.

Step 12: Plot a scatter plot using Hours_Studied on the X-axis, Marks on the Y-axis, and different colors to represent the clusters.

Step 13: Add the axis labels and graph title, then display the scatter plot using plt.show().

Step 14: Stop the program.

Coding and Output:



```
import pandas as pd
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.cluster import AgglomerativeClustering

# Load dataset
data = pd.read_csv("data.csv")

# Fix column names
data.columns = data.columns.str.strip()

# Handle tab-separated data if read as one column
if len(data.columns) == 1 and '\t' in data.columns[0]:
    data = pd.read_csv( filepath_or_buffer: "data.csv", sep="\t")
    data.columns = data.columns.str.strip()

# Select features
X = data[['Hours_Studied', 'Marks']]

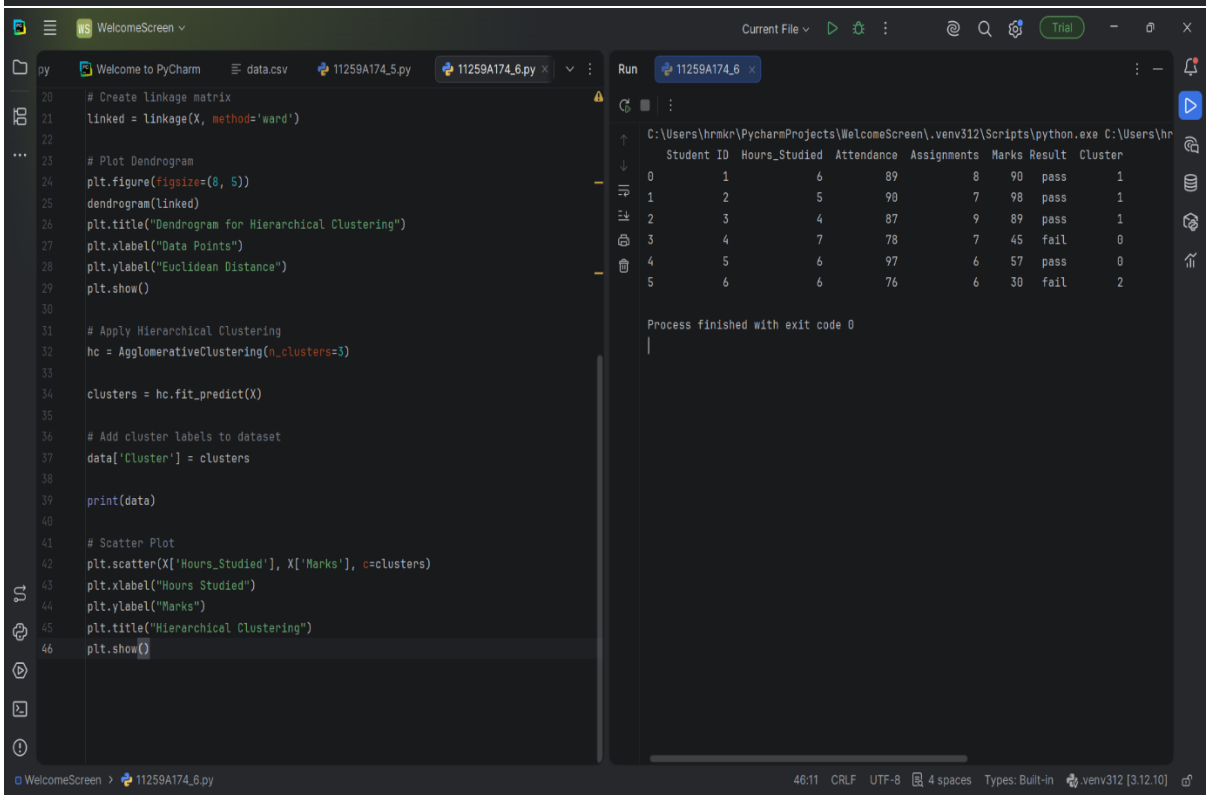
# Create linkage matrix
linked = linkage(X, method='ward')

# Plot Dendrogram
plt.figure(figsize=(8, 5))
dendrogram(linked)
plt.title("Dendrogram for Hierarchical Clustering")
plt.xlabel("Data Points")
plt.ylabel("Euclidean Distance")
plt.show()

# Apply Hierarchical Clustering
hc = AgglomerativeClustering(n_clusters=3)
```

```
C:\Users\hrmkr\PycharmProjects\WelcomeScreen\.venv312\Scripts\python.exe C:\Users\hr
   Student ID  Hours_Studied  Attendance  Assignments  Marks Result  Cluster
0           1              6          89            8     90   pass        1
1           2              5          90            7     98   pass        1
2           3              4          87            9     89   pass        1
3           4              7          78            7     45   fail        0
4           5              6          97            6     57   pass        0
5           6              6          76            6     30   fail        2

Process finished with exit code 0
```



```
# Create linkage matrix
linked = linkage(X, method='ward')

# Plot Dendrogram
plt.figure(figsize=(8, 5))
dendrogram(linked)
plt.title("Dendrogram for Hierarchical Clustering")
plt.xlabel("Data Points")
plt.ylabel("Euclidean Distance")
plt.show()

# Apply Hierarchical Clustering
hc = AgglomerativeClustering(n_clusters=3)

clusters = hc.fit_predict(X)

# Add cluster labels to dataset
data['Cluster'] = clusters

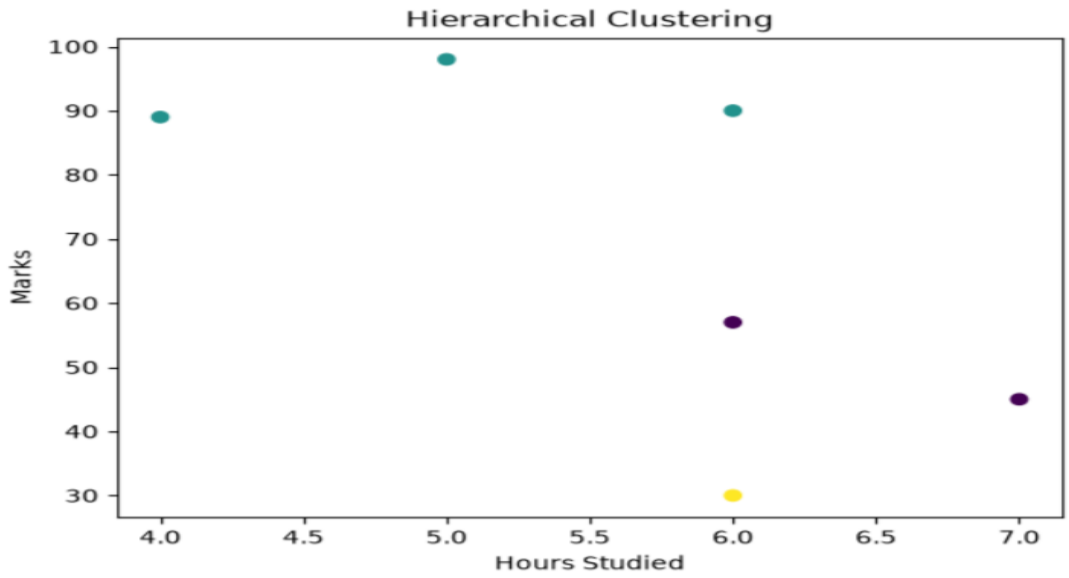
print(data)

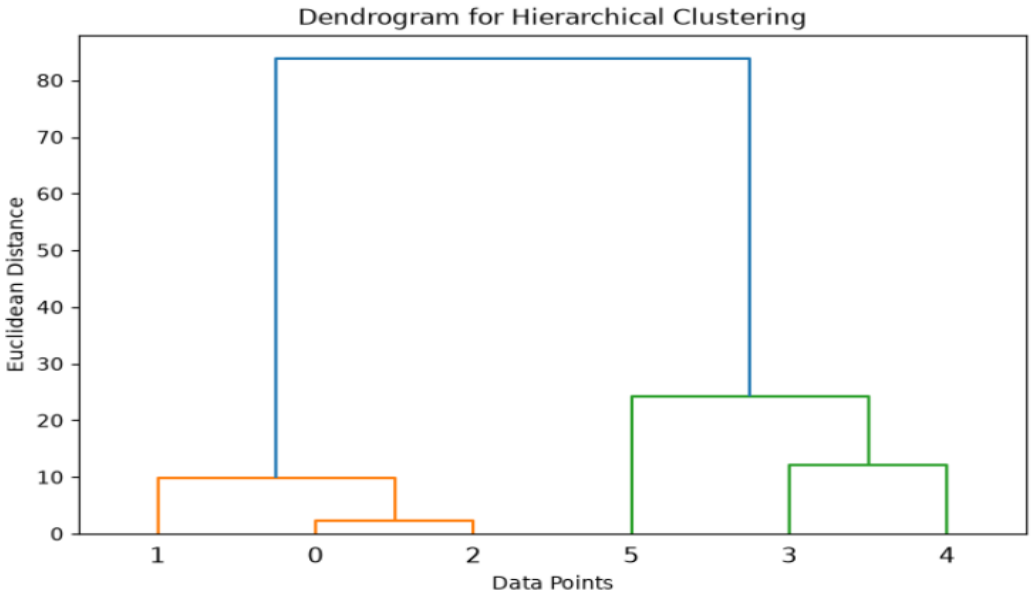
# Scatter Plot
plt.scatter(X['Hours_Studied'], X['Marks'], c=clusters)
plt.xlabel("Hours Studied")
plt.ylabel("Marks")
plt.title("Hierarchical Clustering")
plt.show()
```

```
C:\Users\hrmkr\PycharmProjects\WelcomeScreen\.venv312\Scripts\python.exe C:\Users\hr
   Student ID  Hours_Studied  Attendance  Assignments  Marks Result  Cluster
0           1              6          89            8     90   pass        1
1           2              5          90            7     98   pass        1
2           3              4          87            9     89   pass        1
3           4              7          78            7     45   fail        0
4           5              6          97            6     57   pass        0
5           6              6          76            6     30   fail        2

Process finished with exit code 0
```

Graph:





Result:

The given program has been executed and output has been found out.

Exno:9 Implement Random Forest and Analyze Performance

Aim

To implement the Random Forest Classification algorithm using Scikit-learn to predict students' results based on hours studied, attendance, and assignments, evaluate the model using accuracy, confusion matrix, and classification report, and visualize the importance of each feature using a bar graph.

Algorithm

Step 1: Start the program.

Step 2: Import the required libraries: Pandas, Matplotlib, train_test_split, RandomForestClassifier, accuracy_score, confusion_matrix, and classification_report.

Step 3: Load the dataset from the CSV file data.csv using pd.read_csv().

Step 4: Select Hours_Studied, Attendance, and Assignments as input features and store them in X.

Step 5: Select the Result column as the target variable and store it in y.

Step 6: Split the dataset into training and testing sets using train_test_split(), with 80% training data and 20% testing data, and set random_state=42.

Step 7: Create a Random Forest classification model using RandomForestClassifier(n_estimators=100, random_state=42) with 100 decision trees.

Step 8: Train the model using the training data with model.fit(X_train, y_train).

Step 9: Predict the results for the testing data using model.predict(X_test).

Step 10: Calculate and display the model's accuracy using accuracy_score(y_test, y_pred).

Step 11: Generate and display the confusion matrix using confusion_matrix(y_test, y_pred).

Step 12: Generate and display the classification report using classification_report(y_test, y_pred), including precision, recall, F1-score, and support.

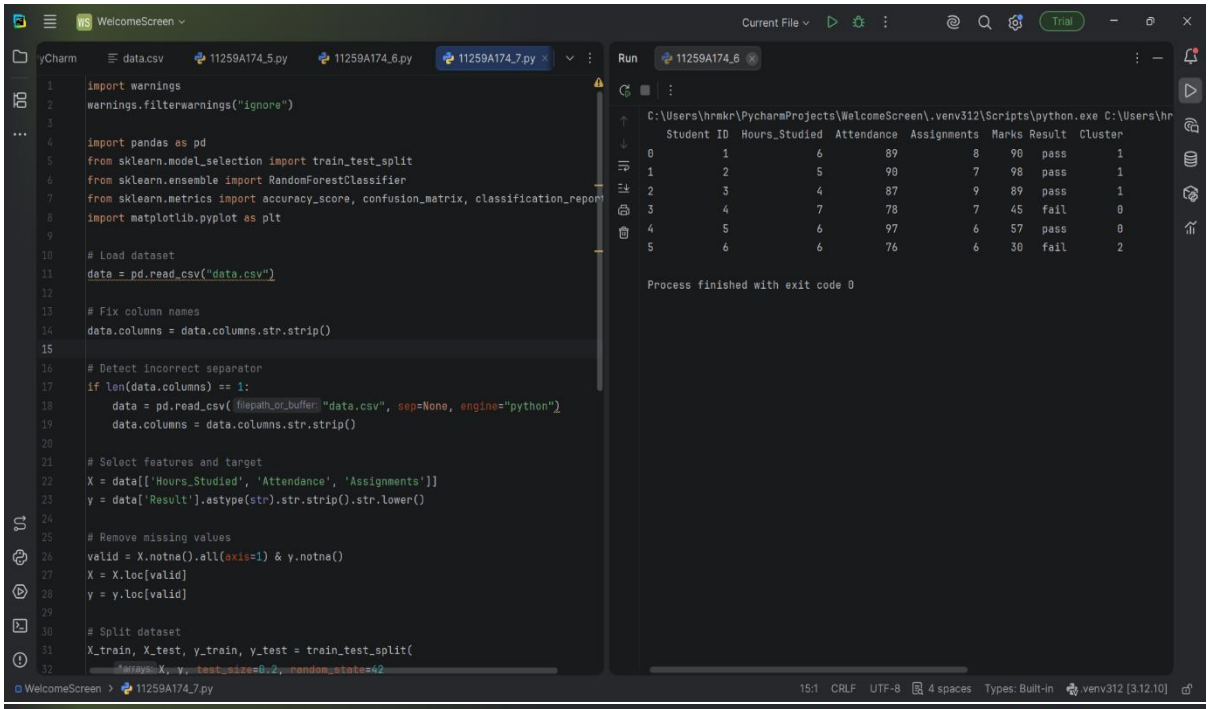
Step 13: Extract the feature importance values using model.feature_importances_.

Step 14: Plot a bar graph showing the importance of each feature, with feature names on the X-axis and importance values on the Y-axis.

Step 15: Add the graph title and display the graph using plt.show().

Step 16: Stop the program.

Coding and Output:



```
import warnings
warnings.filterwarnings("ignore")

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import matplotlib.pyplot as plt

# Load dataset
data = pd.read_csv("data.csv")

# Fix column names
data.columns = data.columns.str.strip()

# Detect incorrect separator
if len(data.columns) == 1:
    data = pd.read_csv( filepath_or_buffer: "data.csv", sep=None, engine="python")
    data.columns = data.columns.str.strip()

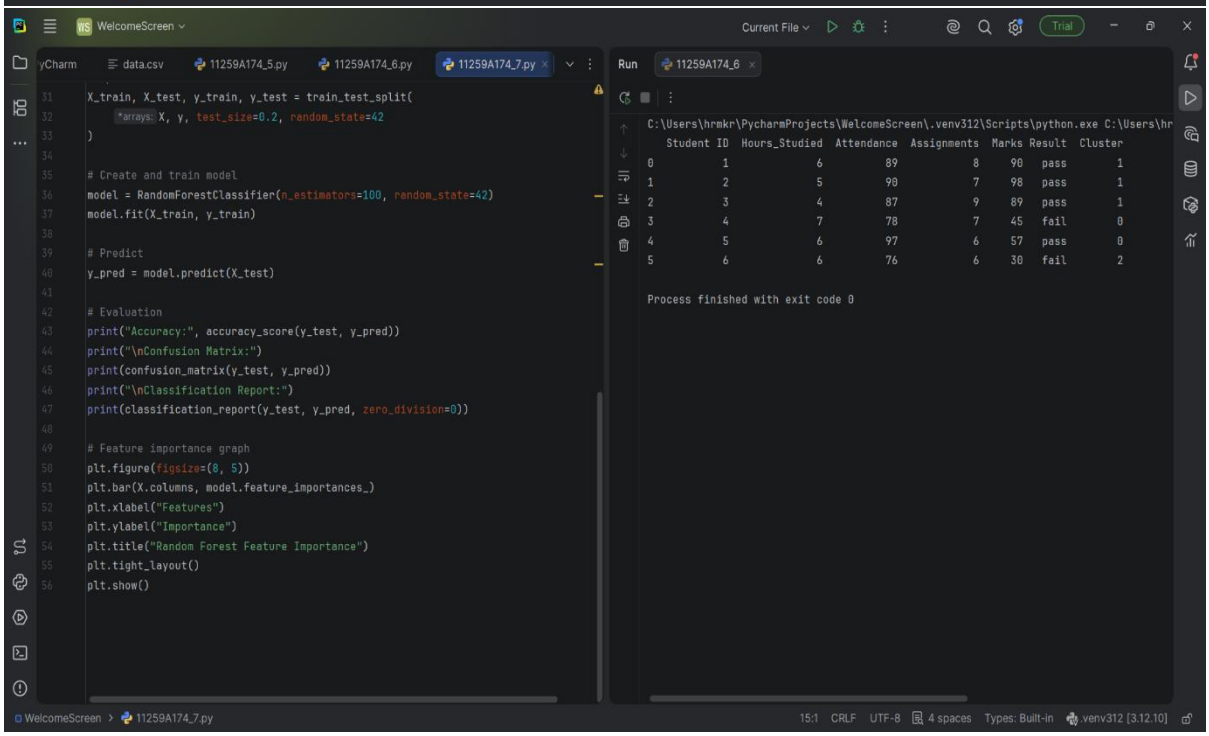
# Select features and target
X = data[['Hours_Studied', 'Attendance', 'Assignments']]
y = data['Result'].astype(str).str.strip().str.lower()

# Remove missing values
valid = X.notna().all(axis=1) & y.notna()
X = X.loc[valid]
y = y.loc[valid]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    *arrays: X, y, test_size=0.2, random_state=42
)
```

```
C:\Users\hrmkr\PycharmProjects\WelcomeScreen\.venv312\Scripts\python.exe C:\Users\hr
   Student ID  Hours_Studied  Attendance  Assignments  Marks Result  Cluster
0           1              6          89            8     90   pass        1
1           2              5          90            7     98   pass        1
2           3              4          87            9     89   pass        1
3           4              7          78            7     45   fail        0
4           5              6          97            6     57   pass        0
5           6              6          76            6     30   fail        2

Process finished with exit code 0
```



```
X_train, X_test, y_train, y_test = train_test_split(
    *arrays: X, y, test_size=0.2, random_state=42
)

# Create and train model
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

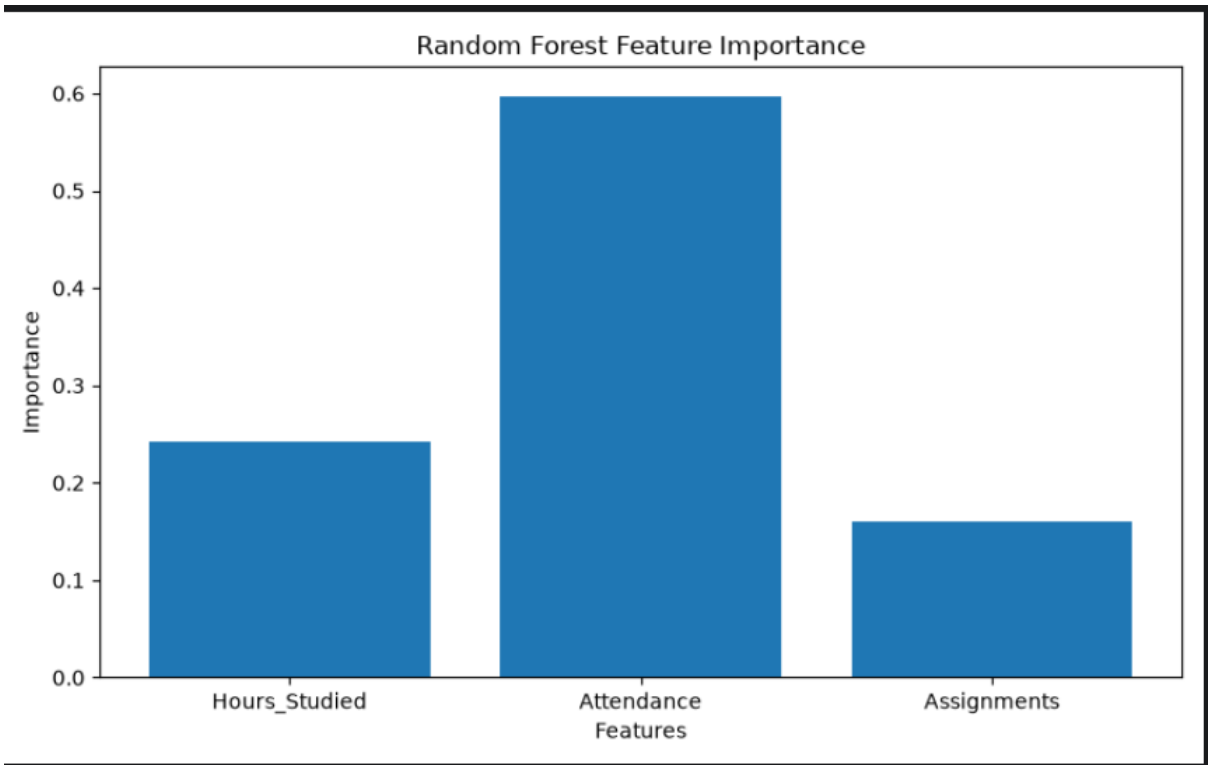
# Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=0))

# Feature importance graph
plt.figure(figsize=(8, 5))
plt.bar(X.columns, model.feature_importances_)
plt.xlabel("Features")
plt.ylabel("Importance")
plt.title("Random Forest Feature Importance")
plt.tight_layout()
plt.show()
```

```
C:\Users\hrmkr\PycharmProjects\WelcomeScreen\.venv312\Scripts\python.exe C:\Users\hr
   Student ID  Hours_Studied  Attendance  Assignments  Marks Result  Cluster
0           1              6          89            8     90   pass        1
1           2              5          90            7     98   pass        1
2           3              4          87            9     89   pass        1
3           4              7          78            7     45   fail        0
4           5              6          97            6     57   pass        0
5           6              6          76            6     30   fail        2

Process finished with exit code 0
```

Graph:



Result:

The given program has been executed and output has been found out.

Exno10: Implement Boosting Technique (AdaBoost) and Analyze Perfeormance

Aim

To implement the AdaBoost Classification algorithm using Scikit-learn to predict students' results based on hours studied, attendance, and assignments, evaluate the model using accuracy, confusion matrix, and classification report, and visualize feature importance using a bar graph.

Algorithm

Step 1: Start the program.

Step 2: Import the required libraries: Pandas, Matplotlib, train_test_split, AdaBoostClassifier, accuracy_score, confusion_matrix, and classification_report.

Step 3: Load the dataset from the CSV file data.csv using pd.read_csv().

Step 4: Select Hours_Studied, Attendance, and Assignments as input features and store them in X.

Step 5: Select the Result column as the target variable and store it in y.

Step 6: Split the dataset into training and testing sets using train_test_split(), with 80% training data and 20% testing data, and set random_state=42.

Step 7: Create an AdaBoost classification model using AdaBoostClassifier(n_estimators=50, random_state=42) with 50 estimators.

Step 8: Train the model using the training data with model.fit(X_train, y_train).

Step 9: Predict the results for the testing data using model.predict(X_test).

Step 10: Calculate and display the model's accuracy using accuracy_score(y_test, y_pred).

Step 11: Generate and display the confusion matrix using confusion_matrix(y_test, y_pred).

Step 12: Generate and display the classification report using classification_report(y_test, y_pred), including precision, recall, F1-score, and support.

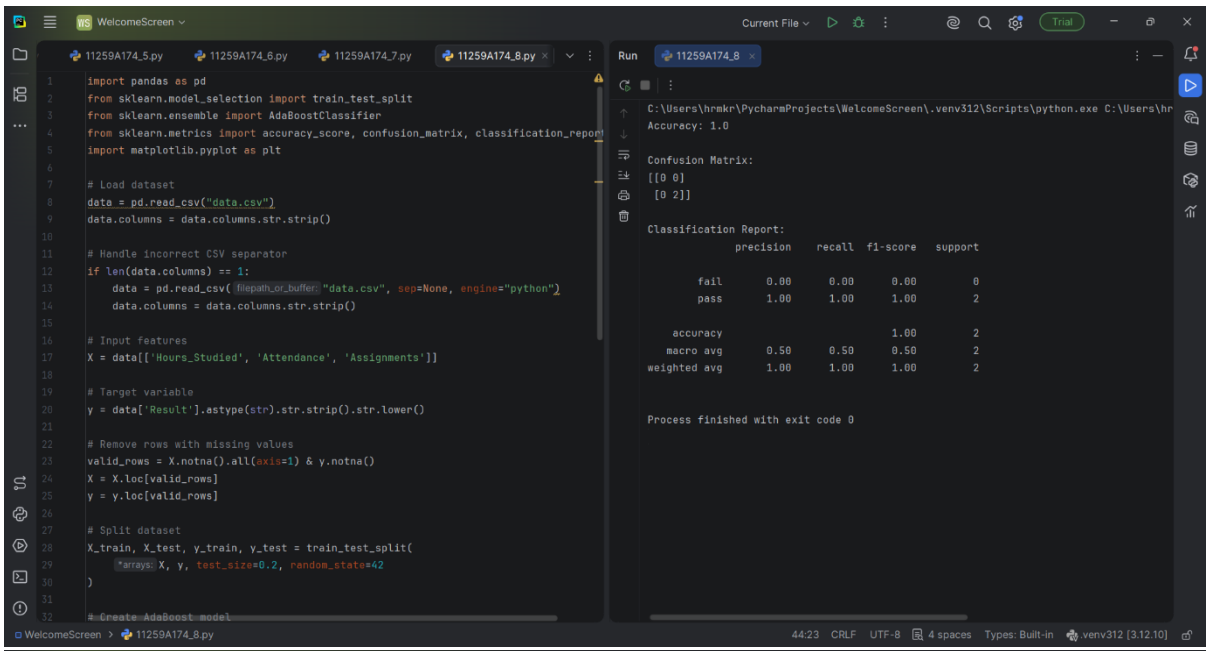
Step 13: Extract the feature importance values using model.feature_importances_.

Step 14: Plot a bar graph showing the importance of each feature, with feature names on the X-axis and importance values on the Y-axis.

Step 15: Add the graph title and display the graph using plt.show().

Step 16: Stop the program.

Coding and Output:



```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import matplotlib.pyplot as plt

# Load dataset
data = pd.read_csv("data.csv")
data.columns = data.columns.str.strip()

# Handle incorrect CSV separator
if len(data.columns) == 1:
    data = pd.read_csv( filepath_or_buffer: "data.csv", sep=None, engine="python")
    data.columns = data.columns.str.strip()

# Input features
X = data[['Hours_Studied', 'Attendance', 'Assignments']]

# Target variable
y = data['Result'].astype(str).str.strip().str.lower()

# Remove rows with missing values
valid_rows = X.notna().all(axis=1) & y.notna()
X = X.loc[valid_rows]
y = y.loc[valid_rows]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    *arrays: X, y, test_size=0.2, random_state=42
)

# Create AdaBoost model
```

```
C:\Users\hrmkr\PycharmProjects\WelcomeScreen\.venv312\Scripts\python.exe C:\Users\hr
Accuracy: 1.0

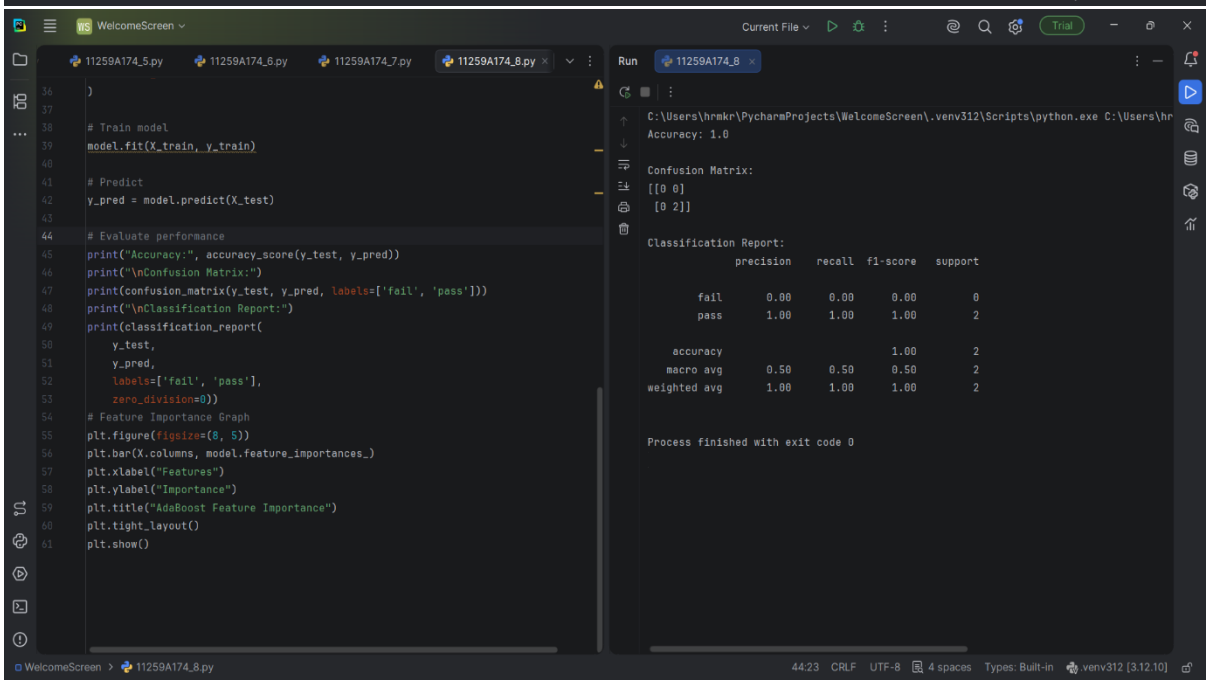
Confusion Matrix:
[[0 0]
 [0 2]]

Classification Report:
              precision    recall  f1-score   support

        fail       0.00      0.00      0.00         0
        pass       1.00      1.00      1.00         2

    accuracy                           1.00         2
   macro avg       0.50      0.50      0.50         2
weighted avg       1.00      1.00      1.00         2


Process finished with exit code 0
```



```
)

# Train model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Evaluate performance
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred, labels=['fail', 'pass']))
print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    labels=['fail', 'pass'],
    zero_division=0))
# Feature Importance Graph
plt.figure(figsize=(8, 5))
plt.bar(X.columns, model.feature_importances_)
plt.xlabel("Features")
plt.ylabel("Importance")
plt.title("AdaBoost Feature Importance")
plt.tight_layout()
plt.show()
```

```
C:\Users\hrmkr\PycharmProjects\WelcomeScreen\.venv312\Scripts\python.exe C:\Users\hr
Accuracy: 1.0

Confusion Matrix:
[[0 0]
 [0 2]]

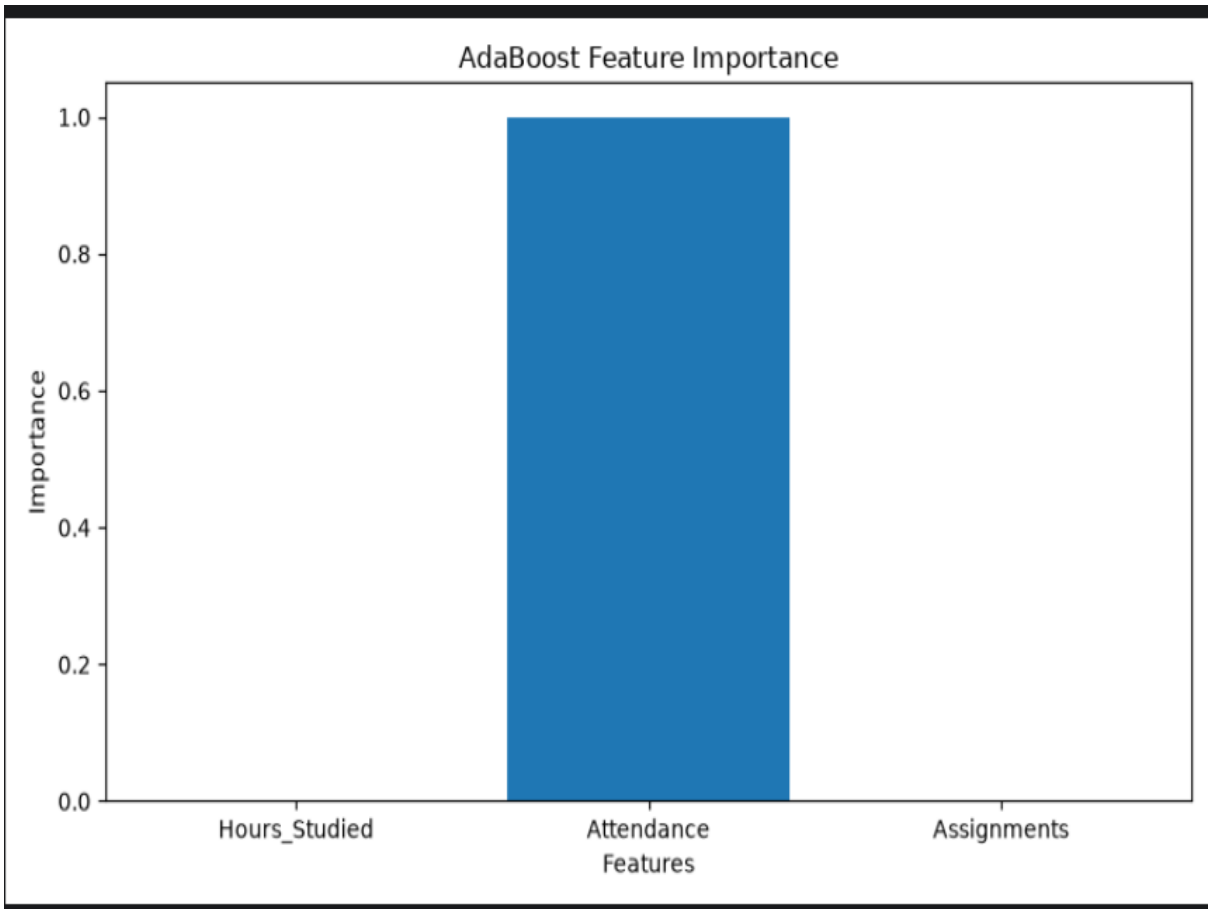
Classification Report:
              precision    recall  f1-score   support

        fail       0.00      0.00      0.00         0
        pass       1.00      1.00      1.00         2

    accuracy                           1.00         2
   macro avg       0.50      0.50      0.50         2
weighted avg       1.00      1.00      1.00         2


Process finished with exit code 0
```

Graph:



Result:

The given program has been executed and output has been found out.

Exno11: Implement a Basic Reinforcement Learning Model (Q-Learning)

Aim:

To implement Q-Learning using a reward matrix to find the optimal path to a goal state and to develop a Neural Network using TensorFlow to predict student results (Pass/Fail) based on hours studied, attendance, and assignments.

Algorithm:

Part A: Q-Learning

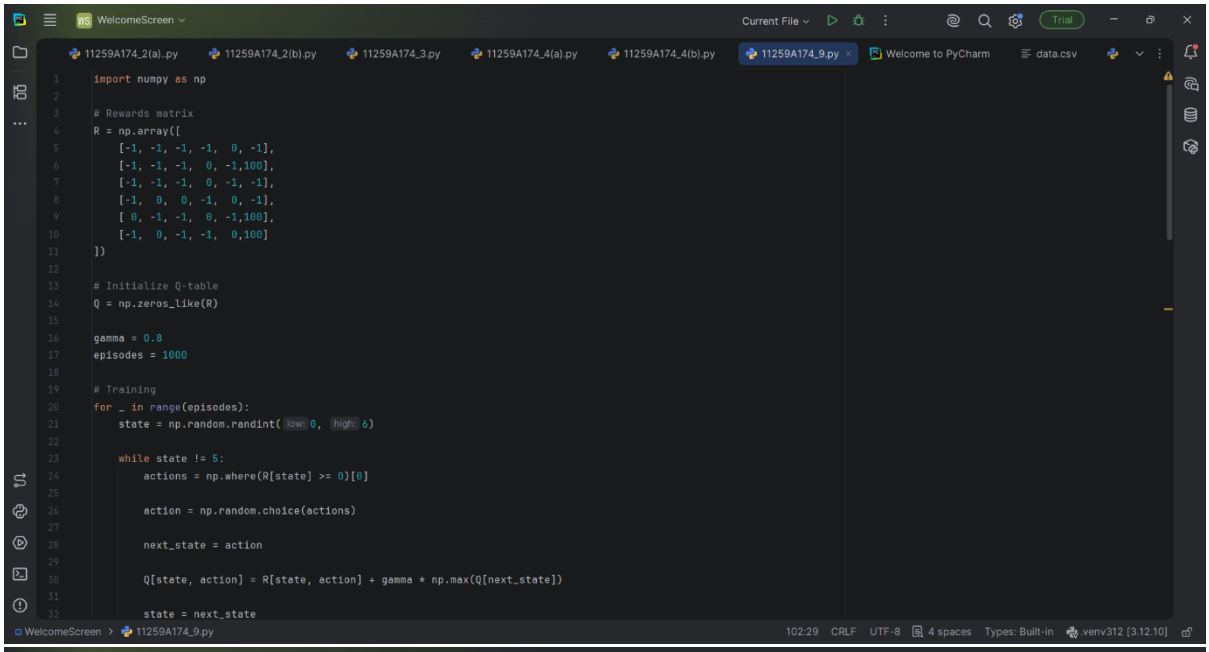
1. Import the NumPy library.
2. Initialize the reward matrix R with rewards for valid state transitions.
3. Initialize the Q-table Q with zeros.
4. Set the discount factor gamma = 0.8 and the number of training episodes to 1000.
5. Randomly select an initial state for each episode.
6. Until the goal state (state 5) is reached:
    - Identify all valid actions from the current state.
    - Randomly select one valid action.
    - Calculate the Q-value using the reward and the maximum future Q-value.
    - Update the current state to the next state.
7. Display the final Q-table.

Part B: Neural Network for Student Result Prediction

1. Import the required libraries: Pandas, TensorFlow, Scikit-learn, and NumPy if needed.
2. Load the dataset from data.csv.
3. Select Hours_Studied, Attendance, and Assignments as input features.
4. Select Result as the target variable.
5. Convert the target labels into numerical values: Fail = 0 and Pass = 1.
6. Split the dataset into training and testing sets using an 80:20 ratio.
7. Apply StandardScaler to normalize the input features.
8. Build a neural network with two hidden layers using ReLU activation and one output layer using sigmoid activation.

9. Compile the model using the Adam optimizer, binary cross-entropy loss, and accuracy metric.
10. Train the model for 50 epochs with 20% of the training data used for validation.
11. Predict the results for the test dataset.
12. Convert predicted probabilities into class labels using a threshold of 0.5.
13. Calculate and display the model's accuracy.

Coding:



```python
import numpy as np

# Rewards matrix
R = np.array([
    [-1, -1, -1, -1,  0, -1],
    [-1, -1, -1,  0, -1,100],
    [-1, -1, -1,  0, -1, -1],
    [-1,  0,  0, -1,  0, -1],
    [ 0, -1, -1,  0, -1,100],
    [-1,  0, -1, -1,  0,100]
])

# Initialize Q-table
Q = np.zeros_like(R)

gamma = 0.8
episodes = 1000

# Training
for _ in range(episodes):
    state = np.random.randint( low: 0,  high: 6)

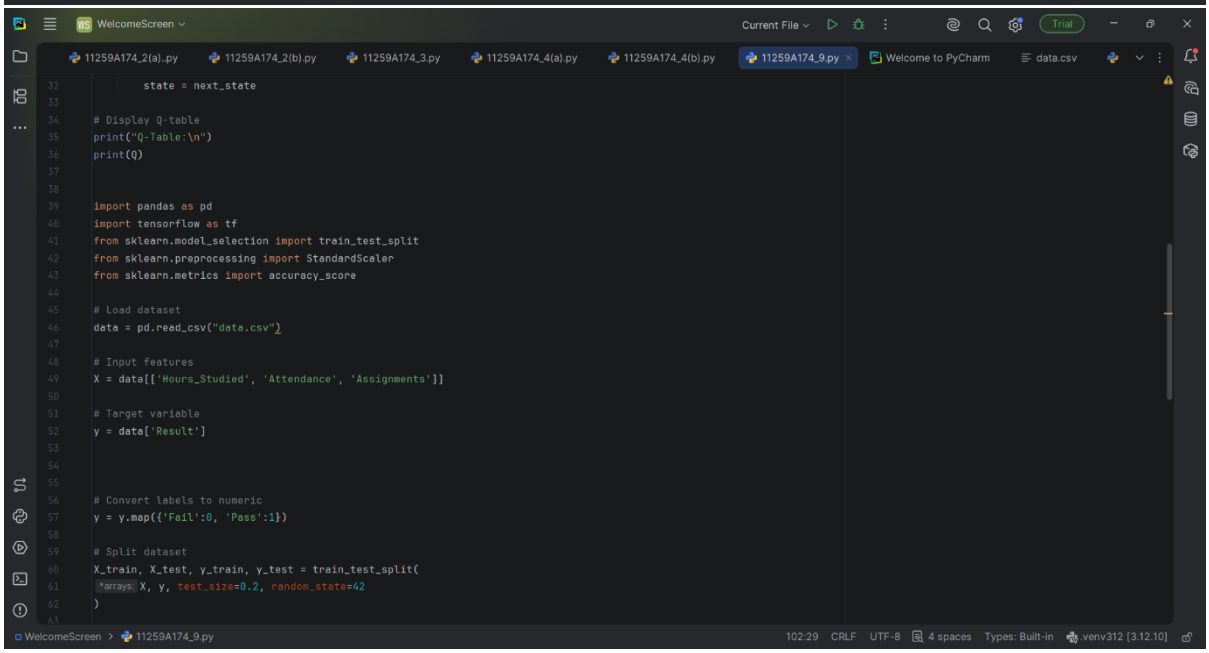
    while state != 5:
        actions = np.where(R[state] >= 0)[0]

        action = np.random.choice(actions)

        next_state = action

        Q[state, action] = R[state, action] + gamma * np.max(Q[next_state])

        state = next_state
```



```python
        state = next_state

# Display Q-table
print("Q-Table:\n")
print(Q)


import pandas as pd
import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

# Load dataset
data = pd.read_csv("data.csv")

# Input features
X = data[['Hours_Studied', 'Attendance', 'Assignments']]

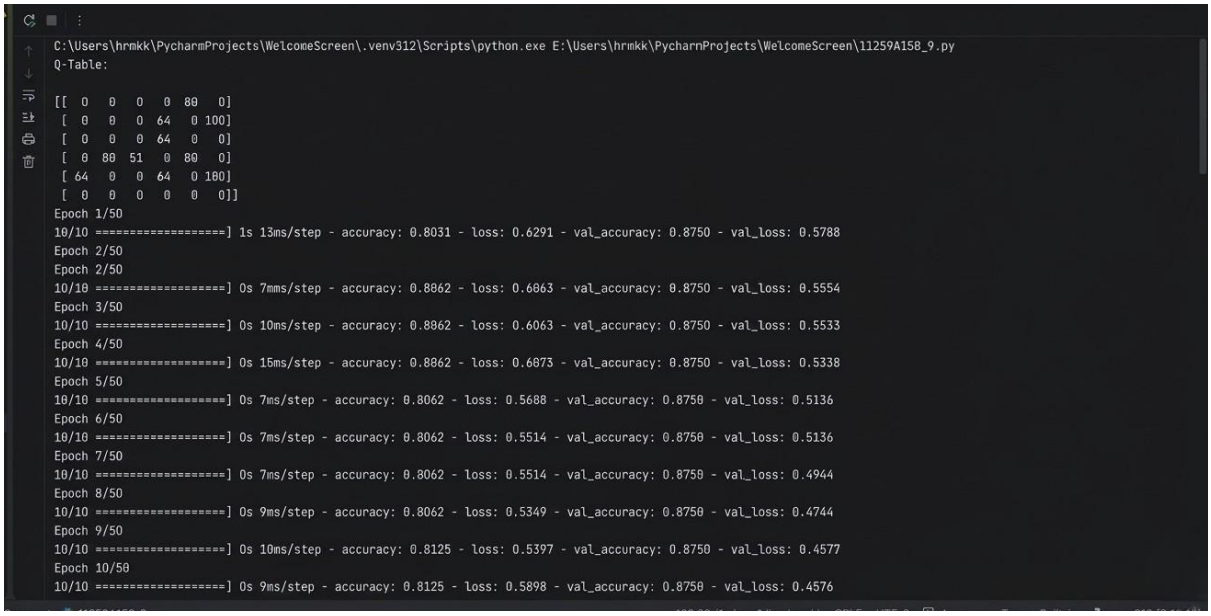
# Target variable
y = data['Result']



# Convert labels to numeric
y = y.map({'Fail':0, 'Pass':1})

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
 *arrays: X, y, test_size=0.2, random_state=42
)
```

Output:



Result:

The given program has been executed and output has been found out.